
Seguretat Informatica (SI)

Seguretat en les aplicacions

René Serral (forked from: Davide Careglio)

Temari

- Tema 1. Introducció
- Tema 2. Criptografia
- Tema 3. Infraestructura PKI
- **Tema 4. Seguretat en les aplicacions**
- Tema 6. Seguretat als sistemes operatius
- Tema 7. Anàlisis forense

Temari

- Tema 1. Introducció
- Tema 2. Criptografia
- Tema 3. Infraestructura PKI
- **Tema 4. Seguretat en les aplicacions**
- Tema 6. Seguretat als sistemes operatius
- Tema 7. Anàlisis forense

Tema 1. Introducció - Índex

- La ciberseguretat
- Objectius
- Amenaces
- Pilars de la seguretat
 - Organització
 - Persones
 - Tecnologia de la informació

Seguretat en les aplicacions

- **Introducció**
- Errors més comuns (segons OWASP)
- Gestió de vulnerabilitats
- Auditories
- Autenticació i Autorització

Introducció

- No existeix un sistema 100% segur
 - Vulnerabilitats teòriques
 - Vulnerabilitats reals (exploits)
- Causes d'una aplicació insegura
 - Vulnerabilitats
 - Estructura organitzativa, processos de desenvolupament, tecnologia
 - Programadors
 - Requisits legals
 - Increment de la connectivitat i la complexitat
 - Canvis de paradigma
 - Cloud
 - IA

Solucions

- Assegurar que un programa segueix unes bones pràctiques usant SDLC i S-SDLC
 - Software Development Life Cycle
- Entendre l'abast de la seguretat que necessita un projecte
- Provar aviat i sovint
- Pensar creativament
- Utilitzar les eines adequades
- Utilitzar codi font quan estigui disponible
- Documentar els resultats de les proves
- Entendre les idiosincràsies de la tecnologia utilitzada

Soluciones

- Assegurar que un programa segueix unes bones pràctiques usant SDLC



Per què cal la programació segura

- Les aplicacions web són la forma més habitual d'atacar un sistema
- La seguretat NO pot ser un problema aliè al programador web
- S'ha de ser conscient dels problemes i de les possibles solucions
- S'ha de suposar que els usuaris NO faran servir l'aplicació únicament de la forma que ha estat dissenyada per l'equip de programació
- No s'ha de confiar en tot el que envien els usuaris, tot i considerant-se que poden estar usant aplicacions segures

Per què cal la programació segura

- Les
- La s
- S'ha
- S'ha
- la f
- No
- que



TRUST NO ONE

ma

s

ment de

ant-se

Seguretat en les aplicacions

- Introducció
- **Errors més comuns (segons OWASP)**
- Gestió de vulnerabilitats
- Auditories
- Autenticació i Autorització

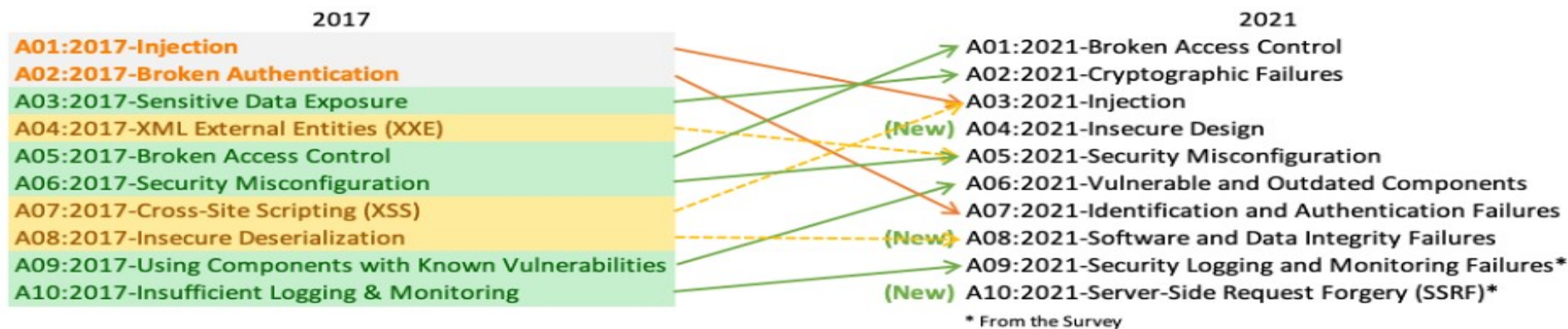
OWASP

- Open Web Application Security Project: <https://owasp.org>
- Fundació sense ànim de lucre
- Participació oberta i gratuïta
- Sense pressions corporatives
- Creada amb la finalitat de determinar i combatre les causes que fan que el software sigui insegur
- Pretén establir mètodes de treball segur a l'hora de desenvolupar aplicacions web

OWASP Top 10 - 2017¹

OWASP Top 10 - 2013	➔	OWASP Top 10 - 2017
A1 – Injection	➔	A1:2017-Injection
A2 – Broken Authentication and Session Management	➔	A2:2017-Broken Authentication
A3 – Cross-Site Scripting (XSS)	➔	A3:2017-Sensitive Data Exposure
A4 – Insecure Direct Object References [Merged+A7]	U	A4:2017-XML External Entities (XXE) [NEW]
A5 – Security Misconfiguration	➔	A5:2017-Broken Access Control [Merged]
A6 – Sensitive Data Exposure	➔	A6:2017-Security Misconfiguration
A7 – Missing Function Level Access Contr [Merged+A4]	U	A7:2017-Cross-Site Scripting (XSS)
A8 – Cross-Site Request Forgery (CSRF)	☒	A8:2017-Insecure Deserialization [NEW, Community]
A9 – Using Components with Known Vulnerabilities	➔	A9:2017-Using Components with Known Vulnerabilities
A10 – Unvalidated Redirects and Forwards	☒	A10:2017-Insufficient Logging&Monitoring [NEW,Comm.]

OWASP Top 10 - 2017¹



Top 10

- Broken Access Control
- Cryptographic Failures
- Injection
- Insecure design
- Security Misconfiguration
- Vulnerable and Outdated Components
- Identification and Authentication Failures
- Software and Data Integrity Failures
- Security Logging and Monitoring Failures
- Server-Side Request Forgery

Broken Access Control

- Passa de la posició 5 (2017) a la 1 (2021)
- El control de accés ha de fer complir la política de seguretat, de manera que els usuaris no puguin actuar fora dels permisos assignats
 - Les restriccions sobre el que poden fer els usuaris sovint no s'apliquen de forma adequada
 - Un atacant pot explotar aquests defectes per accedir a informació no autoritzada, pot modificar o destruir dades o realitzar una funció fora dels límits estipulats

Vulnerabilitats més comuns

- Violació del principi de privilegis mínims (least privilege) o de denegació per defecte (deny by default)
 - L'accés està disponible per qualsevol persona i no depenent d'uns rols, capacitats o usuaris específics
- Eludir les comprovacions del control d'accés
 - P. ex., modificant la URL (i.e., parameter tampering), l'estat de l'aplicació, la pàgina HTML o les sol·licituds a través d'una API
- Permetre veure o editar el compte d'un altre usuari
 - Usant el seu identificador únic (no xifrat o no modificat en la sessió)
- Accés a través d'una API on falten controls d'accés → més sobre això més endavant!
- Elevació de privilegis
 - Actuar com un usuari sense iniciar sessió o actuar com administrador quan s'és un usuari
- Configuració incorrecta de Cross-Origin Resource Sharing (CORS)
 - Un servidor HTTP indica al navegador carregar recursos externs potencialment de tercers
 - Permet l'accés a la API des d'orígens no autoritzats o no confiables

Exemple 1

- Una aplicació usa dades sense verificar en un crida SQL per accedir a la informació d'un compte

```
pstmt.setString(1, request.getParameter("acct"));  
ResultSet results = pstmt.executeQuery( );
```

```
https://example.com/app/accountInfo?acct=notmyacct
```

- Un atacant pot modificar el paràmetre **acct** al navegador per a accedir a la informació de qualsevol compte

Exemple 2

- Un atacant pot anar provant URLs amb accés a pàgines d'administrador

```
https://example.com/app/getappInfo  
https://example.com/app/admin_getappInfo
```

- Un usuari no autenticat NO pot accedir a qualsevol de les pàgines, és un possible punt d'atac
 - Una persona que no és administrador NO pot accedir a la pàgina d'administració
 - Això incrementa la superfície d'atac
-

Prevenió

- Aplicar polítiques de **deny by default** a tots els recursos que no són públics
- Implementar mecanismes de control d'accés i reutilitzar-los durant tot el període d'accés a l'aplicació
 - Minimizar l'us de Cross-Origin Resource Sharing (CORS)
- Aplicar controls sobre les propietats de cada dada, de forma que un usuari no pugui manipular dades dels altres
- Aplicar rate-limit als accessos i usos de les API
- Deshabilitar la opció de llistar el directori del servidor web i assegurar que les metadades i els fitxers de backup no estiguin presents a la pròpia web
- Controlar els logs de fallades d'accés repetidos i fer que saltin alarmes
- En servidors stateful, els identificadors de sessió s'han d'invalidar-se un com acabada la sessió
- En servidors stateless, els tokens poden ser subjectes a vulnerabilitats
- Reduir la duració dels token i usar-los només en cas específic i segur

Cryptographic Failures

- Abans es deia “*Sensitive Data Exposure*”
- Passa de la posició 3 (2017) a la 2 (2021)
- Quan es protegeix de forma incorrecta informació confidencial d'un sistema o en trànsit entre sistemes, com les dades financeres i/o mèdics, targetes bancàries, secrets de negoci, etc.
- Els atacants poden robar o modificar les dades si són protegides dèbilment per a realitzar frauds, robatori d'identitat o altres delictes

Vulnerabilitats més comuns

- Enviament de dades en text pla
 - Per exemple tràfic intern entre servidors de backend
- Utilització d'algorismes de xifrat obsolets o dèbils, alguns usats per defecte quan fallen els actuals
 - e.g, un atacant que deman una connexió oferint només protocols obsolets
- Ús de claus per defecte, dèbils o reutilitzades sense modificació periòdica
- Xifrat no aplicat
 - e.g., no s'utilitzen tots els paràmetres de seguretat a la capçalera HTTP
- Validació incorrecta en la cadena de confiança dels certificats
- Ús d'algorismes de hash obsolets o vulnerables MD5 o SHA-1
- Ús de mètodes de padding obsolets com ara PKCS#1 v1.5

Exemples

- Exemple 1
 - Tenim una aplicació que guarda els números de les targetes de crèdit a una base de dades utilitzant el xifrat automàtic propi de la base de dades
 - Tot i això, quan es fa una consulta, les dades es desxifren automàticament quan s'accedeixen a ells a través del motor de base de dades
 - Això permet a un atacant aprofitar una vulnerabilitat a l'aplicació que permet fer injecció d'SQL, permetent la recuperació de les targetes en clar
- Exemple 2
 - Una aplicació que no utilitza TLS a totes les pàgines
 - Un atacant pot monitoritzar la connexió, forçar un pas de HTTPS a HTTP i robar la cookie de sessió de l'usuari
 - Amb la cookie, l'atacant pot segrestar la sessió autenticada de l'usuari i accedir i modificar les seves dades privades

Prevenió

- Classificar les dades que processa, emmagatzema o transmet una aplicació segons el seu grau de confidencialitat d'acord a la legislació actual
- Descartar tota aquella informació confidencial que no és necessari guardar (eliminar-la incloent de les caches)
- Xifrar tota la informació en trànsit usant protocols segurs com TLS
- Usar els algorismes, protocols y gestors de claus estandarditzats i més actualitzats possible
- Evitar mostrar pàgines d'error per defecte ja que aquestes poden contenir informació confidencial sobre els sistemes
 - Especialment les 4XX i les 5XX
- Emmagatzemar contrasenyes de forma segura
 - De seguida veurem com es fa!

Prevenció: pàgines d'error per defecte

Estado HTTP 500 - Error de Sintaxis en sentencia SQL "SELECT * FROM COMENTARIS WHERE AUTOR='[*]'"

type Informe de Excepción

mensaje Error de Sintaxis en sentencia SQL "SELECT * FROM COMENTARIS WHERE AUTOR='[*]'"

descripción El servidor encontró un error interno que hizo que no pudiera rellenar este requerimiento.

excepción

```
java.lang.RuntimeException: Error de Sintaxis en sentencia SQL "SELECT * FROM COMENTARIS WHERE AUTOR='[*]'"
Syntax error in SQL statement "SELECT * FROM COMENTARIS WHERE AUTOR='[*]'; SQL statement:
SELECT * FROM COMENTARIS WHERE AUTOR='' [42000-188]
    edu.upc.escert.curs.repositori.vulnerable.RepositoriComentaris.getComentarisFromSQL(RepositoriComentaris.java:62)
    edu.upc.escert.curs.repositori.vulnerable.RepositoriComentaris.getComentarisPerAutor(RepositoriComentaris.java:78)
    edu.upc.escert.curs.LlistaComentarisServlet.doGet(LlistaComentarisServlet.java:31)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:621)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:728)
```

nota La traza completa de la causa de este error se encuentra en los archivos de diario de Apache Tomcat/7.0.39.

Apache Tomcat/7.0.39

Prevenió

- Guardar les contrasenyes de forma segura
 - No guardar-les mai en clar
 - Generar un hash per a cada contrasenya de la base de dades (BD)
 - Comparar el hash de la contrasenya introduïda amb el de la BD
- Pot no ser suficient...
 - Raibow tables: <https://crackstation.net/>
Tenir el hash d'una contrasenya simple és com tenir la contrasenya
- Hem d'utilitzar el Salt per evitar atacs per-computats
 - <https://crackstation.net/hashing-security.htm>
 - Concatenem un valor aleatori (Salt) a la contrasenya abans de generar el hash
 - Es guarda el hash com valor associat a l'usuari
 - Es concatena amb la contrasenya introduïda per generar el hash i comparar-lo amb el de la base de dades

Injection

- Passa de la posició 1 (2017) a la 3 (2021)
- S'inclou el Cross-Site Scripting (XSS) a aquesta categoria
- Es produeix quan es passa informació no validada a un intèrpret com a part d'una petició
- El receptor, pensant que són legítims els executa o permet l'accés a dades sense el control d'accés adequada
- Els atacs més comuns d'injecció són a SQL, NoSQL, comandes del SO, LDAP, etc.

Exemple: SQL

```
String query = "SELECT * FROM accounts WHERE custID='" +  
                request.getParameter("id") + "'";
```

- L'atacant pot modificar el valor del paràmetre id al seu navegador i enviar una opció que sempre és certa

`http://example.com/app/accountView?id=' or '1'='1`

- Això canvia el significat de la consulta per a tornar tots els registres de la taula account
- Els atacs més perillosos podrien mofificar o eliminar dades, o inclús invocar procediments emmagatzemats

Exemple: mongoDB

- Molts programadors es pensen que les bases de dades noSQL no són vulnerables a injecció
- La realitat és que ho són tant o més
 - Tenen stored procedures
 - Poden invocar codi Javascript en segons quins casos

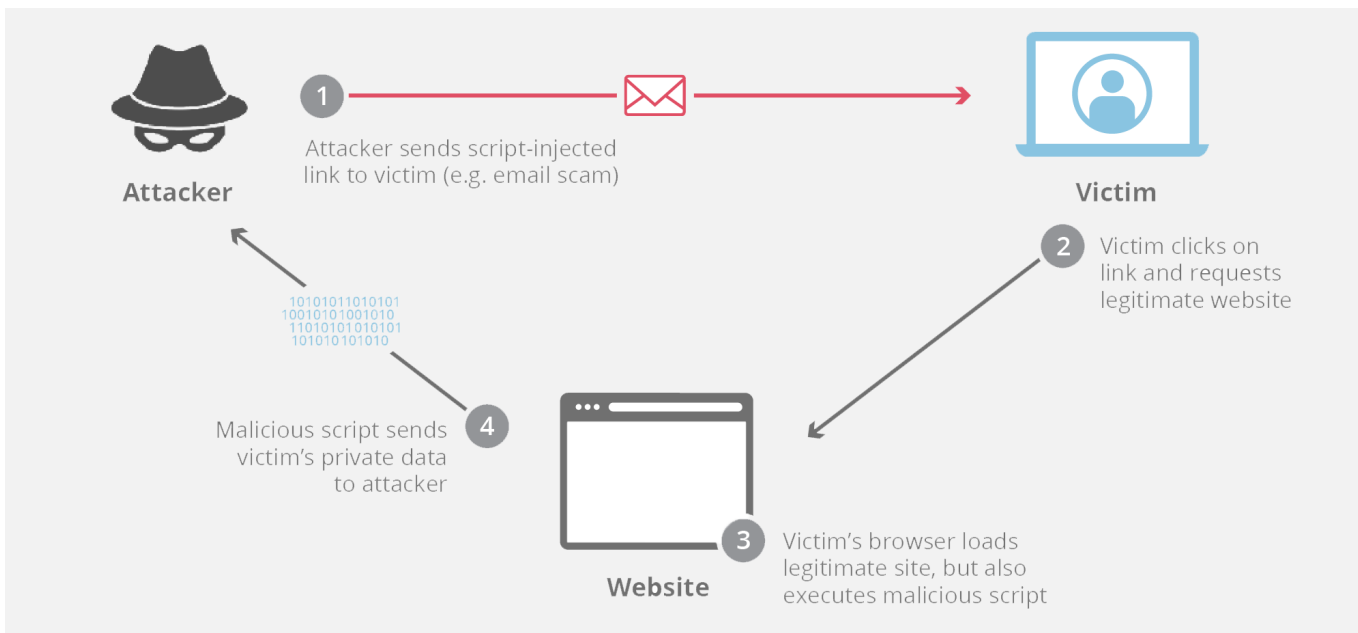
```
db.collection.find({ $where: function() {  
    return (this.product == $productData)  
}});
```

```
db.collection.find({ $where: function() {  
    return (this.product == 'xx'; sleep (10000))  
}});
```

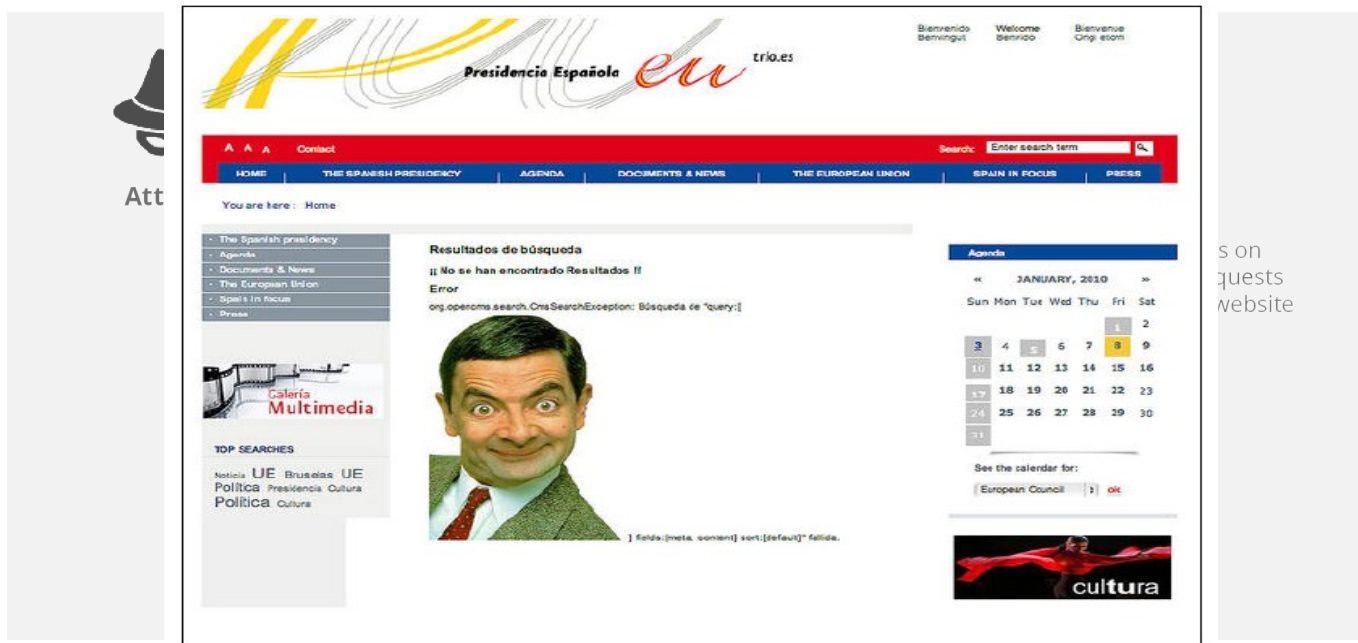
Cross-Site Scripting (XSS)

- XSS permet als atacants executar scripts al navegador de la víctima
 - Robatori de sessions d'usuari
 - Manipular llocs web
 - Redirecció de l'usuari a llocs maliciosos
- Passa quan una aplicació
 - inclou dades no confiables a un enllaç sense validació o escapament adequats
 - actualitza una pàgina web existent amb dades proporcionades per l'usuari utilitzant una API que permeti executar HTML o JavaScript

Cross-Site Scripting (XSS)



Cross-Site Scripting (XSS)



Prevenió

- La opció preferida és usar una API segura, que eviti totalment l'ús de l'interpret o proporcioni una interfície parametritzada i validada
- Utilitzar la validació d'entrada positiva o white-list del **costat del servidor**
- Utilizar LIMIT i altres controls SQL sobre el número i tipus de consultes per a evitar la divulgació massiva de registres en caso de injecció SQL
- Usar la sintaxis de escapament específica per a evitar l'ús de caràcters especiales que validen una consulta
- Filtrar segons la semàntica de l'aplicació, per exemple
 - **L'Hospitalet** com a nom vàlid de ciutat
 - **important** pot ser un comentari vàlid si l'aplicació permet un camp amb text enriquit, però no si no és així

Insecure design

- Nova categoria del 2021
- El disseny insegur és una categoria àmplia que representa diferents debilitats
 - Expressades com disseny de control no present o ineficax
- Disseny i implementació són aspectes molt diferents
 - Un disseny segur pot seguir tenint una implementació insegura
 - Un disseny insegur no es pot “salvar” amb una implementació segura
- Molts cops un disseny és insegur per no haver creat els perfils de risc, pel que no s’ha inclòs una solució segura des de o

Exemple 1

- Un sistema ofereix la possibilitat de recuperar les credencials d'un usuari a través de preguntes i respostes, per exemple

Indica la població on vas nèixer

- No es pot confiar en les preguntes i respostes com evidència d'identitat, pot haver-hi molta gent que les conegui
- Aquest tipus de recuperació de credencials estan prohibides actualment
- Aquest codi s'ha d'eliminar i utilitzar una alternativa més segura

Exemple 2

- Un lloc web es dedica a la venda de GPU d'alta gamma
- No té un sistema de protecció contra bots de compra massiva
- Els atacants compren tot l'stock automàticament i el revenen als llocs de subhastes
- Pel que els compradors només poden accedir a preus més elevats pel producte
- La web acaba tenint mala reputació
- Es necessita un «anti-bot» que denegui determinades compres segons regles específiques
 - P.e., limitar el rate de compres de producte

Prevenió

- Integrar verificacions de plausibilitat a cada nivell de l'aplicació
 - Des de el frontend fins al backend
- Establir i utilitzar un SDLC amb professionals en Application Security per ajudar al disseny
- Utilitzar un model d'amenaces per a l'autenticació crítica, el control d'accés, la lògica i l'intercanvi de claus
- Preparar tests de validació per a cada bloc i per a la seva integració i comprovar que són resistents al model d'amenaces
 - Preparar tests tant per al cas d'ús legítim com per a l'il·legítim
- Segregar i aïllar els grups d'usuaris que comparteixen un accés comú
- Limitar el consum de recursos per part de l'usuari o servei

Security Misconfiguration

- Passa de la posició 6 (2017) a la 5 (2021)
- S'inclou ara la categoria del 2017 anomenada XML External Entities (XXE)
- Tot el que implica una configuració de seguretat dèbil, mal implementada, o directament inexistent

Vulnerabilitats més comuns

- Falta del nivel de seguridad adecuado en cualquier parte de las aplicaciones o permisos configurados incorrectamente
- Hay funciones habilitadas o instaladas que son innecesarias
 - P. e., puertos, servicios, páginas, cuentas o privilegios innecesarios
- Cuentas, claves o contraseñas predeterminadas aún habilitadas
- El software está desactualizado o es vulnerable
- El servidor no envía cabeceras o directivas de seguridad, o no están configurados con valores seguros
- Las últimas funciones de seguridad en los sistemas actualizados están deshabilitadas o no configuradas de forma segura
- El gestor de errores revela a los usuarios información innecesaria

Exemples

- Exemple 1
 - El servidor incorpora aplicacions demo instal·lades pel fabricant i no s'han eliminat del servidor en producció
 - Aquestes aplicacions demo tenen errors de seguretat coneguts que els atacants utilitzen per a comprometre el servidor
 - Si per exemple, una d'aquestes aplicacions és la consola d'administració i no s'han modificat els comptes predeterminats, l'atacant pot iniciar sessió amb les contrasenyes predeterminades i prendre el control del servidor
- Exemple 2
 - La llista de directoris no està deshabilitada al servidor
 - Un atacant pot descobrir aquest accés i trobar i descarregar les classes Java compilades
 - Amb tècniques de descompilació i enginyeria inversa pot descobrir el codi font i analitzar-lo per trobar vulnerabilitats

Security Misconfiguration - XML External Entities (XXE)

- Consisteix en aconseguir que una aplicació amb un parser XML rep o executi un codi maliciós
 - Un XML pot contenir un *Document Type Definition (DTD)* que, entre d'altres coses, permet la definició d'entitats XML referenciades a través d'una URI
 - D'aquesta manera, a l'executar un parser XML mal configurat es pot colar una URI a documents o objectes confidencials, o forçar connexions externes malicioses saltant les regles del firewall
- D'aquesta manera es pot aconseguir
 - Arxius interns
 - Escaneig de ports interns
 - Execució remota de codi
 - Atacs de denegació de servei

Examples

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<foo>&xxe;</foo>
```

Aconseguim accedir al fitxer d'usuaris del sistema Linux/Unix

```
<!ENTITY xxe SYSTEM "https://192.168.1.1/private">]>
```

S'aconsegueix informació sobre una direcció interna privada

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY lol "lol">
  <!ELEMENT lolz (#PCDATA)>
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
  <!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
  <!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
  <!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
  <!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
  <!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<lolz>&lol9;</lolz>
```

Atac DoS
(billion laughs)

Prevenió

- Configurar una tasca per a revisar els nivells de seguretat i actualitzar aplicacions i instal·lar patches de seguretat
- Revisar els permisos d'emmagatzemament al núvol
- Mantenir en producció una plataforma mínima sense atributs, components, documentació ni exemples innecessaris, eliminant funcions i frameworks no utilitzats
- Definir i configurar els entorns de desenvolupament, control de qualitat i producció de forma idèntica però amb diferents credencials en cada entorn
- Definir un procés automatitzat per a verificar la efectivitat de les configuracions i ajustos en tots els entorns
 - Deshabilitar la opció DTD per a evitar XXE
 - Validar els arxius XML de entrada
 - Actualitzar els parsers

Vulnerable and Outdated Components

- En versions anteriors s'anomenava “*Using Components with Known Vulnerabilities*”
- Passa de la posició 9 (2017) a la 6 (2021)
- Tot el que fa referència a l'ús intern o extern d'aplicacions, protocols, llibreries, SO, ... que contenen vulnerabilitats i que no estan actualitzats

Problemes més comuns

- Quan no es coneixen les versions de tots els components que es fan servir (tant pel costat del client com del servidor)
 - Això inclou els components directes així com les dependències
- Quan el software és vulnerable, incompatible o no actualitzat
 - SO, servidor web, l'administrador de bases de dades, les API i les llibreries
- Quan no es busquen vulnerabilitats regularment
- Quan no s'asseguren les configuracions dels components
- Quan no es comprova la compatibilitat de les llibreries modificades, actualitzades o parchejades
- Quan no es repara o s'actualitza la plataforma base, els frameworks i les dependències de manera oportuna

Example

- Vulnerabilitat a Zoom des de la versió 5.03 a 5.12 per a Windows

Posted by [Mitja Kolsek](#) on [July 09, 2020](#)

Remote Code Execution Vulnerability in Zoom Client for Windows (0day)

by Mitja Kolsek, the Opatch Team

Earlier this week a security researcher shared a remote code execution "0day" vulnerability in Zoom Client for Windows with our team. The vulnerability allows a remote attacker to execute arbitrary code on victim's computer where Zoom Client for Windows (any currently supported version) is installed by getting the user to perform some typical action such as opening a document file. No security warning is shown to the user in the course of attack.

The researcher (who wants to keep their identity private) stated that they did not report the vulnerability to Zoom either directly or through a broker, but would not object to us reporting it to Zoom.

[Update 7/13/2020: Zoom only took one (!) day to issue a new version of Client for Windows that fixes this vulnerability, which is remarkable. We have reviewed their fix and can confirm that it efficiently resolves the vulnerability. With an official vendor fix available to all users, we made our micropatches for this issue PRO-only according to our [guidelines](#). Meanwhile, after issuing micropatches for this issue targeted at Zoom Client for Windows versions 5.0.3 to 5.1.2, we noticed a lot of our users being on all of these versions despite Zoom's highly persistent update mechanism. We had expected most users to be on version 5.1.2., but this indicates many users may still be on even older Zoom Client versions. We therefore ported our micropatch to the remaining supported versions of Zoom Client: 5.0.0., 5.0.1, and 5.0.2. We're now covering all vulnerable supported clients.]

Prevenió

- Eliminar les dependències, funcions, components, arxius i documentació innecessàries o no utilitzades
- Realitzar un inventari continu de les versions dels components (p. e., frameworks, llibreries) i les seves dependències, utilitzant eines com OWASP Dependency Check, retire.js, etc.
- Supervisar continuament fonts com la Common Vulnerability and Exposures (CVE) i la National Vulnerability Database (NVD) per a les vulnerabilitats als components
- Obtenir components de fonts oficials a través d'enllaços segurs, usant paquets signats per a reduir la possibilitat d'incloure un component maliciós modificat
 - Relacionat amb Software and Data Integrity Failures
- Monitoritzar les llibreries i els components que no es mantenen o no creen patches de seguretat per a versions antigues

Identification/Authentication Failures

- Passa de la posició 2 (2017) a la 7 (2021)
- Abans es deia “Broken authentication”
- Tot el que fa referència als problemes relacionats amb el procés de login, gestió de sessions i credencials

Vulnerabilitats més comuns

- Fer servir contrasenyes dèbils Solarwinds123
- Admetre contrasenyes per defecte com admin, password, 1234
- Permetre atacs automàtics i de força bruta
- Usar mètodes de recuperació de contrasenya poc segurs
 - Relacionat amb Insecure design
- Autenticació simple sense control multifactor
- Mantenir la base de dades de les contrasenyes amb poca seguretat
 - Relacionat amb Cryptographic Failures
- Manteniment de les sessions amb IDs únics o exposats
- Exposar l'ID de les sessions a la URL

Prevenió

- Assegurar que les connexions HTTP facin servir un canal segur i xifrat
 - Usar HTTPs i TLS quan s'envien usuaris i contrasenyes i durant tota la connexió per a que ningú vegi les cookies de sessió
- Usar Cookie “Secure”
 - Per a que les cookies s'enviïn només si s'està usant HTTPs
- Usar HTTPOnly com atribut
 - Per a que les cookies no es vegin a Javascript

Prevenió

- Implementar un verificador de contrasenyes dèbils
 - Top 10,000 worst passwords
<https://github.com/danielmiessler/SecLists/tree/master/Passwords>
 - Usar les recomanacions sobre complexitat, longitud i duració de les contrasenyes
<https://pages.nist.gov/800-63-3/sp800-63b.html#memsecret>
- Implementar tècniques d'autenticació multifactor per a evitar
 - Atacs massius i automàtics
 - Robatori de credencials (credential stuffing)
 - Força bruta

Prevenió

- Evitar el problema del “Session Fixation”
 - Es produeix quan s’entra a una sessió i aquesta es manté amb el mateix ID del login i sense temporitzador
 - Un atacant podria tornar a entrar passat un temps ja que la sessió segueix oberta
 - Un usuari podria passar la URL a un altre usuari i aquest accedir amb les credencials del primer usant l’ID del login
- Per a evitar aquest problema
 - Usar un administrador de sessió segur del costat del servidor que generi un nou ID de sessió aleatori amb alta entropia després de l’inici de sessió
 - Els ID de sessió no han d’estar a la URL, han d’emmagatzemar-se de forma segura i invalidar-se després de tancar sessió o passat un temps d’inactivitat

Software and Data Integrity Failures

- Nova categoria en 2021
- S'inclou el "Insecure deserialization" a aquesta categoria
- Tot el que fa referència als problemes de seguretat relacionats amb fer suposicions quan es realitzen actualitzacions de software, de dades crítiques i de canals de CI/CD sense verificar la integritat

Problemes més comuns

- Quan una aplicació es basa en complements, llibreries o mòduls obtinguts de fonts, repositoris i/o CDN que no són de confiança
- Quan un canal d'integració i desenvolupament continuos (CI/CD) és poc segur i un atacant pot obtenir accés no autoritzat, introduir codi maliciós o comprometre el sistema
- Quan es realitzen actualitzacions automàtiques i aquestes es descarreguen sense suficient verificació d'integritat
 - Els atacants podrien carregar les pròpies actualitzacions per distribuir-les i executar-les a totes les instal·lacions
 - Recordar el cas del software Orion de SolarWinds (Tema 1)

Exemple

- Actualització maliciosa de SolarWinds
- L'empresa que desenvolupa el software Orion tenia processos de construcció i actualització segurs i amb control d'integritat
- Tot i així, es van poder saltar (recordar la contrasenya)
- Durant diversos mesos, la empresa va distribuir una actualització amb codi maliciós a més de 18.000 clients
- Es tracta d'una de les infraccions d'aquesta naturalesa més transcendents i significatives de la història

Failures Insecure deserialization

- És causat quan al processar i deserialitzar unes dades que ha enviat un atacant, s'aconsegueix introduir un objecte que aconsegueix modificar el comportament de l'aplicació
- Aquest objecte pot ser
 - El contingut d'una Remote Procedure Call
 - Una cookie
 - Tokens d'una API
 - Una crida a un web service
- Nota:
 - Serialitzar: és convertir un objecte en un format (exemples: binari, XML/JSON) que es poden guardar o enviar a un perifèric (stdout, xarxa, etc.)
 - Deserialitzar: procés invers, es rep una dada serialitzada o es reconverteix en l'objecte original

Failures Insecure deserialization – exemple

- Un forum usa serialització d'objectes PHP per a guardar “super-cookies” que contenen l’ID dels usuaris, els seus rols, les contrasenyes i altres estats

```
a:4:{i:0;i:132;i:1;s:7:"Mallory";i:2;s:4:"user";  
i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

- Un atacant pot canviar l'objecte serialitzat per a donar-se privilegis d'administrador

```
a:4:{i:0;i:1;i:1;s:5:"Alice";i:2;s:5:"admin";  
i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

Prevenção

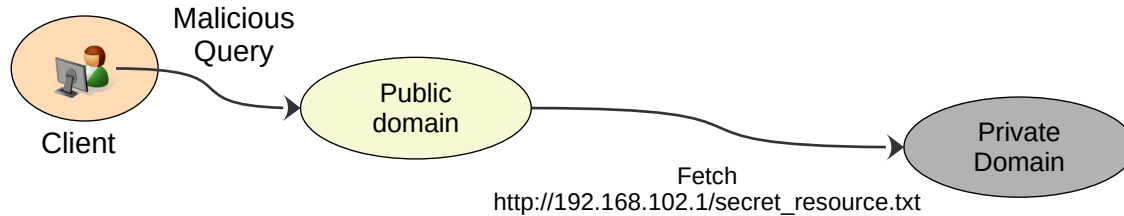
- Utilitzar firmes digitals o mecanismes similars per a verificar que el software o les dades proveniu de la font de confiança
- Assegurar que les llibreries i dependències usen repositoris de confiança
- Assegurar que hi hagi un procés de revisió dels canvis de codi i configuració per a minimitzar la possibilitat de que s'introdueixin codis maliciosos o s'obrin bretxes de seguretat
- Assegurar que el canal de CI/CD tingui segregació i control d'accés adequats per a garantir la integritat del codi
- No acceptar objectes serialitzats de fonts no confiabels o usar verificació d'integritat o firma digital per a detectar alteracions

Server Side Request Forgery

- Nova categoria al 2021
- Si bé no hi ha molts atacs que se n'aprofiten els que funcionen tenen més impacte que la mitja
- Pot afectar a qualsevol aplicació web que obté recursos remots sense validar la URL proporcionada pel client

Problemes més comuns

- Avui dia hi ha una gran quantitat d'aplicacions web que permeten la descàrrega de fitxers de forma remota
- Això implica que quan un sistema té accés a recursos de forma única, un atacant maliciós podria demanar la descàrrega d'aquests fitxers



Exemples¹

- **Escenari #1:** Realitzar un port scan a servidors interns, mirant el temps de resposta de connexions a ports concrets
- **Escenari #2:** Accedir a recursos locals ([file:///etc/shadow](#)) o ([http://localhost:6969](#))
- **Escenari #3:** Accedir a metadades de serveis cloud: [http://169.254.169.254/](#)
- **Escenari #4:** Atacs contra serveis interns

¹ https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/

Prevenió

- A nivell de xarxa:
 - Segmentar la xarxa per evitar l'accés a recursos locals de forma arbitrària
 - Aplicar la **política d'accés mínim** i «**deny by default**»
- A nivell d'aplicació:
 - Sanititzar i validar les dades enviades per l'usuari!
 - Aplicar un esquema d'URL on només les que el compleixen s'envien
 - No permetre les redireccions HTTP
 - Compte amb la consistència d'URL per evitar atacs del tipus **DNS rebinding**¹

Seguretat en les aplicacions

- Introducció
- Errors més comuns (segons OWASP)
- **Gestió de vulnerabilitats**
- Auditories
- Autenticació i Autorització

Common Platform Enumeration



Es necessiten processos d'inventariat altament confiables i automatitzables

- CPE proporciona:
 - Un format estàndard llegible per màquina per a codificar números de productes i plataformes
 - Un conjunt de procediments per a comparar noms
 - Un llenguatge per a construir "declaracions d'aplicabilitat" que combinen noms de CPE amb operadors lògics simples

Examples

 **cpe:2.3:o:microsoft:windows_10:1703:*:*:*:*:***

[Hide Matching CPE\(s\)](#) ▲

- *cpe:2.3:o:microsoft:windows_10:1703:*:*:*:*:*
- *cpe:2.3:o:microsoft:windows_10:1703:*:*:*:*:x64:*
- *cpe:2.3:o:microsoft:windows_10:1703:*:*:*:*:x86:*

 **cpe:2.3:o:microsoft:windows_10:1709:*:*:*:*:**

[Hide Matching CPE\(s\)](#) ▲

- *cpe:2.3:o:microsoft:windows_10:1709:*:*:*:*:*
- *cpe:2.3:o:microsoft:windows_10:1709:*:*:*:*:arm64:*
- *cpe:2.3:o:microsoft:windows_10:1709:*:*:*:*:x64:*
- *cpe:2.3:o:microsoft:windows_10:1709:*:*:*:*:x86:*

Common Vulnerabilities and Exposures

- Identificadors que s'usen per a indicar una vulnerabilitat
- Definit i mantingut per MITRE Corporation
 - <https://cve.mitre.org>

CVE-ID	
CVE-2019-1359	Learn more at National Vulnerability Database (NVD) • CVSS Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings • CPE Information
Description	
A remote code execution vulnerability exists when the Windows Jet Database Engine improperly handles objects in memory, aka 'Jet Database Engine Remote Code Execution Vulnerability'. This CVE ID is unique from CVE-2019-1358.	
References	
Note: References are provided for the convenience of the reader to help distinguish between vulnerabilities. The list is not intended to be complete.	
• MISC:https://portal.msrc.microsoft.com/en-US/security-guidance/advisory/CVE-2019-1359	
Assigning CNA	
Microsoft Corporation	
Date Record Created	
20181126	Disclaimer: The record creation date may reflect when the CVE ID was allocated or reserved, and does not necessarily indicate when this vulnerability was discovered, shared with the affected vendor, publicly disclosed, or updated in CVE.
Phase (Legacy)	
Assigned (20181126)	

Common Vulnerability Score System (CVSS)

- Definit a partir de l'any 2004
- Es tracta d'un sistema de puntuació dissenyat per a proveir un mètode obert i estàndard que permet estimar l'impacte derivat de vulnerabilitats
- CVSS es troba sota la custòdia de Forum of Incident Response and Security Teams (FIRST), però es tracta d'un estàndard completament obert
- <https://www.first.org/cvss>
- Actualment versió 3.1



CVSS – exemple

- CVE-2019-1359

CVSS v3.1 Severity and Metrics:

Base Score: 7.8 HIGH

Vector: AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H

Impact Score: 5.9

Exploitability Score: 1.8

Attack Vector (AV): Local

Attack Complexity (AC): Low

Privileges Required (PR): None

User Interaction (UI): Required

Scope (S): Unchanged

Confidentiality (C): High

Integrity (I): High

Availability (A): High

National Vulnerability Database (NVD)

- El NIST disposa d'una base de dades amb totes les vulnerabilitats que han aparegut des de 2002
 - <https://nvd.nist.gov>
- Permet la consulta directa o bé descarregar totes les dades en format XML
- S'actualitza diàriament
 - Agrega els CVSS a posteriori, entre 8 i 48 hores després normalment



National Vulnerability Database - Example

🚩 CVE-2019-1359 Detail

Current Description

A remote code execution vulnerability exists when the Windows Jet Database Engine improperly handles objects in memory, aka 'Jet Database Engine Remote Code Execution Vulnerability'. This CVE ID is unique from CVE-2019-1358.

[+ View Analysis Description](#)

Severity

CVSS Version 3.x

CVSS Version 2.0

CVSS 3.x Severity and Metrics:



NIST: NVD

Base Score: **7.8 HIGH**

Vector: CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H

NVD Analysts use publicly available information to associate vector strings and CVSS scores. We also display any CVSS information provided within the CVE List from the CNA.

Note: NVD Analysts have published a CVSS score for this CVE based on publicly available information at the time of analysis. The CNA has not provided a score within the CVE List.

QUICK INFO

CVE Dictionary Entry:

CVE-2019-1359

NVD Published Date:

10/10/2019

NVD Last Modified:

08/24/2020

Source:

Microsoft Corporation

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

Hyperlink

<https://portal.msrc.microsoft.com/en-US/security-guidance/advisory/CVE-2019-1359>

Resource

Patch **Vendor Advisory**

Gestió de vulnerabilitats – Aplicació

- Per a poder usar tota aquesta informació és necessari fer un treball previ
- Inventari d'actius que permeti saber:
 - Quin són els actius de la organització
 - On estan
 - Quines funcions desenvolupen
 - En quin estat es troben

o-days

- Les vulnerabilitats o-day són aquelles que s'han descobert sense que el fabricant del producte ho sàpiga
- Aquestes vulnerabilitats es poden posar a disposició de la comunitat i permeten la creació d'exploits
- Durant el període de temps disponible entre aquest descobriment i que el fabricant solucione el problema, hi ha clarament una criticitat alta

Seguretat en les aplicacions

- Introducció
- Errors més comuns (segons OWASP)
- Gestió de vulnerabilitats
- **Auditories**
- Autenticació i Autorització

Auditories

- Segons la Real Acadèmia de la Llengua Espanyola:
 - L'auditoria consisteix en la revisió sistemàtica d'una activitat o d'una situació per a avaluar el compliment de les regles o criteris objectius als que les mateixes ha de ser sotmeses
- En el nostre cas consisteix en revisar la seguretat i la integritat d'aplicacions o servidors i avaluar les possibles vulnerabilitats trobades

Tècniques

- Inspeccions i revisions manuals
- Modelat d'amengaces
- **Revisió de codi**
- Proves d'intrusió

Revisió de codi

- És una bona idea realitzar un anàlisi del codi que escrivim
 - Millor si ho fa algú extern
 - O un procés ...
- Es pot analitzar el codi de manera
 - Estàtica (caixa blanca)
 - Dinàmica (en execució)

Anàlisi estàtic

- Permet comprovar:
 - Que es segueixen bones pràctiques de programació
 - Vulnerabilitats típiques
- Se li poden escapar errors de disseny propis del llenguatge o l'aplicació en si mateixa
- Segons la fase del SDLC
 - Codificació
 - Proves



Eines per l'Anàlisi estàtic

- PMD
- SonarQube
- Coverity
- FxCop: per Microsoft .NET
- FinBugs: cerca patrons d'error en Java
- FlawFinder: per c/c++
- RIPS: per PHP
- Bandit: per Python

Anàlisis dinàmic

- Permet comprovar el comportament de l'aplicació en un entorn d'execució real
- Poden aparèixer errors no detectats en l'anàlisi estàtic
- Eines
 - El debugger de sempre
 - OWASP Zap
 - W3AF

Tècniques

- Inspeccions i revisions manuals
- Modelat d'amengaces
- Revisió de codi
- **Proves d'intrusió**

Què són les proves d'intrusió?

- Atacs controlats contra la infraestructura
- Amb l'objectiu de trencar el sistema i poder veure com reacciona a inputs esperats
- Té per objectiu atacar a tota la superfície d'atac
 - Tot i que de cops s'acota l'abast per temes pressupostaris
- Serveixen per quantificar la seguretat d'una infraestructura

Proves d'intrusió

- Es poden fer de diverses formes
 - Caixa Negra
 - Caixa Gris
 - Caixa Blanca

Caixa Negra

- Sense informació prèvia de la infraestructura
- Dóna molta llibertat

Caixa Gris

- Informació mínima
- Permet dirigir millor l'atac

Caixa Blanca

- Amb tota la informació
- Accés al codi font
- Permet un anàlisi més profund

Proves d'intrusió

- Avantatges
 - Pot ser ràpid i barat
 - Requereixen un coneixement relativament menor que una revisió de codi font
 - Comproven si el codi està realment exposat
- Inconvenients
 - Massa tard segons el SDLC
 - Només proven l'impacte frontal

Són legals les proves d'intrusió?

- Article 197bis, accés a dispositius informàtics
- 1. Qui, per qualsevol medi o procediment, vulnerant les mesures de seguretat establertes per impedir-ho, i **sense estar degudament autoritzat**, accedeixi o faciliti a un altre accés al conjunt o part d'un sistema informàtic o es mantingui en ell en contra de la voluntat de qui tingui un dret legítim a excloure'l, serà castigat amb pena de presó d'entre 6 mesos i 2 anys
- 2. Qui, per mitjà de la utilització d'artificis o instruments tècnics, i **sense estar degudament autoritzat**, intercepti transmissions no públiques de dades informàtiques, incloses les emissions electromagnètiques dels mateixos, serà castigat en una pena de presó d'entre 3 mesos a dos anys, o multa de tres a dotze mesos

Passos de la prova d'intrusió

- **Information gathering**
- Escaneig
- Anàlisi manual o automàtic
- Explotació/obtenció d'evidències
- Realització d'un Informe

Information gathering

- A l'hora de fer una auditoria és important obtenir el màxim d'informació possible sobre els servidors o serveis a auditar
- No només ens hem d'informar, sinó també de buscar «coneixements indeguts» com ara pot ser la estructura interna de la xarxa, arxius que haurien d'estar ocults ...
- És a dir, volem trobar informació que no hauríem de tenir

Exemples

- A través de peticions DNS als servidors és possible obtenir molta informació
 - Número de servidors de correu / DNS (redundància)
 - Serveis externs (caching, correu)
 - Subdominis
 - Servidors virtualitzats
- Hi ha varies eines que permeten realitzar aquestes peticions
 - nslookup
 - dig
 - fierce

```
[dync-35-224:~ davidecareglio$ nslookup -q=mx upc.edu
Server:          147.83.30.71
Address:         147.83.30.71#53
```

Non-authoritative answer:

```
upc.edu mail exchanger = 5 alt1.aspmx.l.google.com.
upc.edu mail exchanger = 5 alt2.aspmx.l.google.com.
upc.edu mail exchanger = 10 alt3.aspmx.l.google.com.
upc.edu mail exchanger = 10 alt4.aspmx.l.google.com.
upc.edu mail exchanger = 1 aspmx.l.google.com.
```

Authoritative answers can be found from:

```
[dync-35-224:~ davidecareglio$ nslookup -q=ns upc.edu
Server:          147.83.30.71
Address:         147.83.30.71#53
```

Non-authoritative answer:

```
upc.edu nameserver = ns1.upc.edu.
upc.edu nameserver = ns1.cesca.cat.
upc.edu nameserver = ns2.upc.edu.
upc.edu nameserver = ns2.cesca.cat.
upc.edu nameserver = fina-r-sip.upc.edu.
```

Authoritative answers can be found from:

```
ns1.upc.edu      internet address = 147.83.0.1
ns2.upc.edu      internet address = 147.83.0.2
```

Information gathering – Google hacking

- És una tècnica que utilitza el cercador de Google amb paràmetres específics que permeten filtrar per URL o certes cadenes que ens poden permetre descobrir arxius interns de servidors que han sigut indexats
- Google proporciona operadors especials
 - operador:terme
 - Sense espais i l'operador en minúscules
- El terme pot ser
 - Una paraula
 - Una frase (entre cometes)
- Hi ha operadors que s'han d'utilitzar sols
 - Comencen amb la paraula reservada ALL
- Si hi ha un error sintàctic s'entén com un terme més a buscar

Information gathering – Google hacking

- Eines d'automatització de Google hacking
 - FoundStone Sitedigger
 - Apollo
 - Athena
 - Wikto
- Google Hacking Database (GHDB)
 - Colecció de tècniques de Google hacking
 - <https://www.exploit-db.com/google-hacking-database>

Information gathering – Metadades

- Molts documents contenen informació extra
- Les metadades poden proporcionar informació
 - Usuaris
 - Localització
 - IPs
 - Software
- Eines
 - stat
 - pdftinfo
 - file
 - <https://metashieldclean-up.elevenpaths.com>

Information gathering – Open Source Intelligence

- Hi ha eines que ens ajuden a buscar registres oberts digitals
- Són eines legals que amb la cerca adequada ens donen resultats que poden ajudar un atac posterior
- Exemples:
 - Maltego – Cerca genèrica
 - Shodan
 - exifTool

Information Gathering

- Es recomana buscar aparicions del subjecte que està sent auditat a:
 - Deep Web – compte!
 - Pastebin
 - Telegram
 - Shodan

Passos de la prova d'intrusió

- Information gathering
- **Escaneig**
- Anàlisi manual o automàtic
- Explotació/obtenció d'evidències
- Realització d'un Informe

Escaneig

- La primera cosa que volem esbrinar és la superfície d'atac des de l'exterior
- Així, volem saber quines IP i ports estan oberts i quins serveixen per oferir més serveis a través seu
- Tenim moltes eines que ens ajuden a això

Escaneig - nmap

- Eina executable per la línia de comandes, molt potent per realitzar escaneig de xarxes, ports, SO, serveis, versions, etc...

```
nmap -v -A scanme.nmap.org
```

Escaneig detallat amb detecció de SO, versions, scripts i traceroutes

```
nmap -F scanme.nmap.org
```

Escaneig ràpid que només detecta els ports oberts o filtrats



Example – nmap

```
dync-35-224:~ davidecareglio$ sudo nmap -O -sSU scanme.nmap.org
Starting Nmap 7.93 ( https://nmap.org ) at 2023-04-12 12:00 CEST
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.17s latency).
Not shown: 989 closed tcp ports (reset), 285 closed udp ports (port-unreach), 714
open|filtered udp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
25/tcp    filtered smtp
53/tcp    open  domain
80/tcp    open  http
135/tcp   filtered msrpc
139/tcp   filtered netbios-ssn
161/tcp   filtered snmp
445/tcp   filtered microsoft-ds
593/tcp   filtered http-rpc-epmap
9929/tcp  open  nping-echo
31337/tcp open  Elite
123/udp   open  ntp
Aggressive OS guesses: Linux 2.6.32 or 3.10 (93%), Linux 4.4 (93%), Linux 2.6.32
(92%), Linux 2.6.32 - 2.6.35 (91%), Linux 2.6.32 - 2.6.39 (91%), Linux 4.0 (90%),
Linux 2.6.32 - 3.0 (90%), Linux 3.11 - 4.1 (89%), Linux 3.2 - 3.8 (89%), Linux 5
.0 - 5.4 (89%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 22 hops

OS detection performed. Please report any incorrect results at https://nmap.org/s
ubmit/ .
Nmap done: 1 IP address (1 host up) scanned in 1377.32 seconds
```

Passos de la prova d'intrusió

- Information gathering
- Escaneig
- **Anàlisi manual o automàtic**
- Explotació/obtenció d'evidències
- Realització d'un Informe

Anàlisi manual o automàtic

- Un cop esbrinada la superfície d'atac, s'ha d'analitzar el sistema per descobrir vulnerabilitats
- Forma Automàtica
 - Usant eines disponibles que facin el test de penetració amb els atacs més comuns
- Forma Manual
 - Un expert amb coneixements actualitzats (documentar-se i estar al dia són factors importants!), intuïció i creativitat és necessari per a fer proves més exhaustives i completes

Eines d'anàlisi automàtic

- **Wfuzz:** eina molt coneguda i molt configurable que permet atacar serveis mitjançant fuzzing
- **sqlmap:** per a realitzar tests de penetració a bases de dades
- **OWASP ZAP:** permet l'anàlisi d'aplicacions web, fent de proxy entre el navegador i el servei a auditar
- **OpenVAS:** eina que permet escanejar una infraestructura, detectar els serveis que ofereix i automatitzar la explotació de les vulnerabilitats

Passos de la prova d'intrusió

- Information gathering
- Escaneig
- Anàlisi manual o automàtic
- **Explotació/obtenció d'evidències**
- Realització d'un Informe

Explotació

- Un cop detectades les (possibles) vulnerabilitats es poden explotar
- Compte!
 - Ha d'haver-hi un acord amb el responsable del sistema
 - El sistema es pot caure pel que s'han d'acordar finestres de test
- Millor demanar un entorn de testing

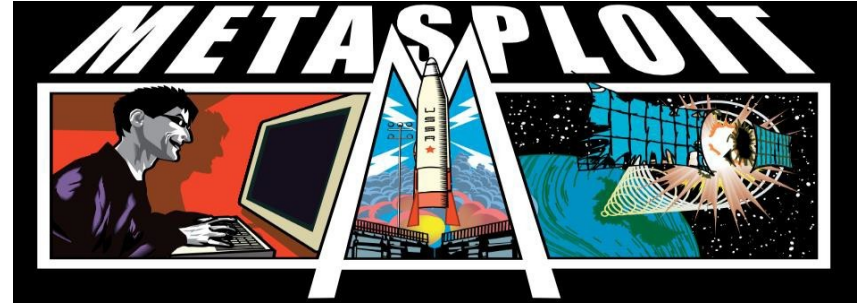
Explotació

- El primer pas per a explotar una vulnerabilitat és mirar si ja existeix algun exploit conegut
- Algunes són relativament senzilles de comprovar:
 - Un arxiu que hauria d'estar ocult es pot accedir
 - Podem injectar SQL a un camp concret
- De cops n'hi ha altres que requereixen d'exploits més complets, però també en podem trobar a exploit-db
 - <https://www.exploit-db.com/>



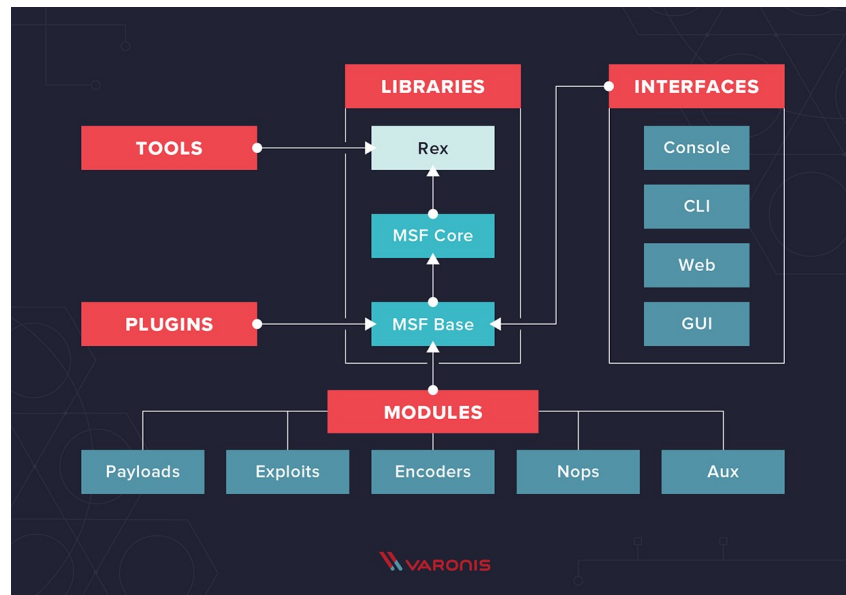
Funcionalitats – Metasploit

- Desenvolupar exploit
- Executar exploit a màquines locals o remotes
- Escanejar màquines buscant vulnerabilitats
- Recollir informació sobre vulnerabilitats



El Metasploit Framework (MSF)

- Llibreries
- Interfícies
- Descobrir o importar informació sobre vulnerabilitats trobades (scanning)
- Mòduls addicionals gratuïts o de pagament
 - Exploits (més de 2000 disponibles) que permeten accedir d'alguna forma a un sistema
 - Payloads (més de 550 disponibles) que permeten executar algun codi a un sistema



Seguretat en les aplicacions

- Introducció
- Errors més comuns (segons OWASP)
- Gestió de vulnerabilitats
- Auditories
- **Sistemes de gestió dels drets d'accés d'usuaris**
- Autenticació i Autorització

Sistemes de gestió d'accés a recursos

- Des del punt de vista de disseny i de seguretat és imprescindible tenir un control d'accés als recursos compartits molt granular
- Aquest sistema hauria de ser
 - Fàcil de mantenir
 - Prou ric per proporcionar les polítiques de seguretat necessàries per l'empresa
- Què un usuari tingui accés a un sistema no vol dir que hagi de tenir accés a tots els seus recursos
 - Administradors vs usuaris regulars
 - Distinció entre tipus d'usuaris o el seu rol



Què és Autorització?

- Sistema que valida que un usuari autenticat (o no) té privilegis (o no) per accedir a un recurs
- Si un recurs és públic
 - S'ha de garantir que tothom hi pot accedir
- Si un recurs és privat
 - S'ha de garantir que l'usuari està autenticat
 - S'ha de garantir que l'usuari té els privilegis adequats



Sistemes de gestió dels drets d'accés d'usuari

- **Access Control Lists (ACL)**
- Role Based Access Control (RBAC)
- Attribute Based Access Control (ABAC)
- Policy Based Access Control (PBAC)

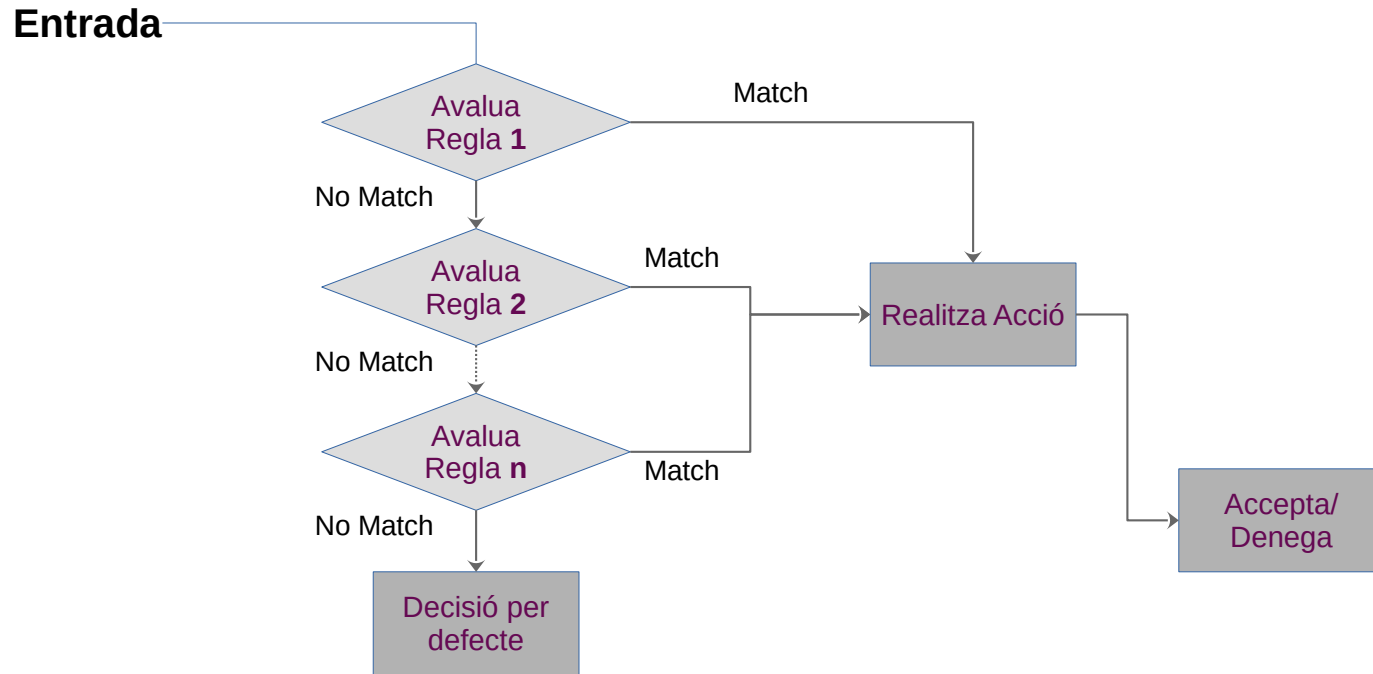


Access Control Lists (ACL)

- Són llistes de regles
 - Generalment relacionades amb firewalls, però també:
 - Llistat de regles per llançar alarmes en un EDR o SIEM
 - Accés a recursos en Windows
 - ... Bàsicament qualsevol entorn en el que es pugui indicar una seqüència de regles i un comportament per defecte
- Amb una política per defecte
- Amb un veredict final
 - Acceptat
 - Denegat
- De cops incorpora també un acció a realitzar
 - Posar una entrada al log
 - Processar la petició, ...



Com funcionen les ACL



Inconvenients de les ACL

- L'anàlisi seqüencial fa que si n'hi ha moltes poden ser un coll d'ampolla (podem tenir-ne desenes de milers)
- Normalment no tenen estat
 - Cada petició ha de passar per tota la llista
- Manca de context
 - Llista declarativa d'accions
- No hi ha dinamisme
 - Llista de regles estàtica → Pot ser modificada per agents externs



Sistemes de gestió dels drets d'accés d'usuari

- Access Control Lists (ACL)
- **Role Based Access Control (RBAC)**
- Attribute Based Access Control (ABAC)
- Policy Based Access Control (PBAC)



Role Based Access Control¹

- Com ACL és un mecanisme per restringir l'accés a recursos
- Es defineixen «papers» (Rols) als que s'assignen els usuaris
 - Cada rol té un conjunt de privilegis particulars
- Es pot complementar amb
 - Discretionary Access Control (DAC)
 - Mandatory Access Control (MAC) → Generalment no es fan servir amb RBAC

¹ <https://www.imperva.com/learn/data-security/role-based-access-control-rbac/>

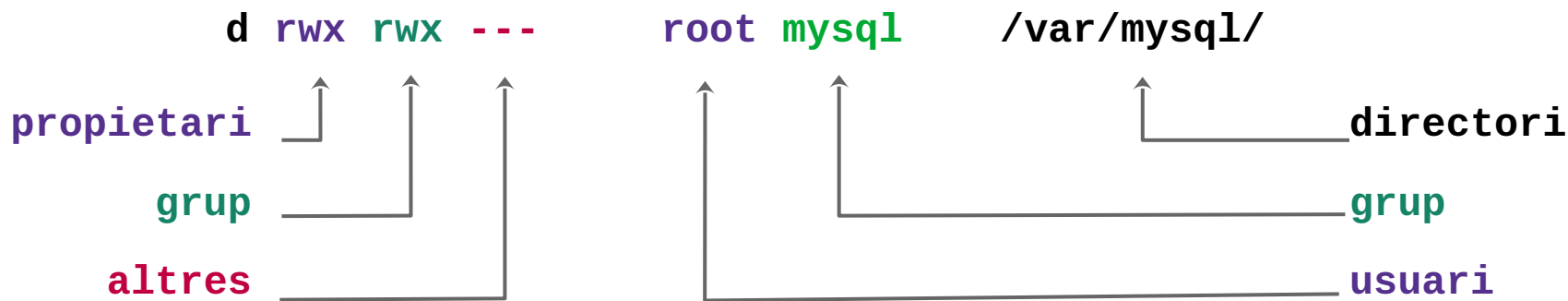
Role Based Access Control

- Els rols i els permisos es poden entendre com una matriu
- La granularitat dependrà del sistema
- Exemple

Rol	Email	Xarxa Interna	Base de dades	Dades del projecte 1	Dades del projecte 2
Participant projecte 1	✓	✓	✓	✓	✗
Participant projecte 2	✓	✓	✗	✗	✓
Administrador	✓	✓	✓	✓	✓
Convidat	✓	✗	✗	✗	✗

Discretionary Access Control (DAC)

- Declaració de permisos per al recurs
 - Per exemple en forma d'ACL
- És la típica definició



Mandatory Access Control (MAC)

- Els privilegis es determinen a través de condicions més complexes
- Determina la llista de condicions que s'han de complir per poder realitzar l'acció
 - L'usuari pot escriure a `/var/mysql` si ho fa amb
 - L'aplicació mysql
 - Entre les 17:00 i les 20:00
- Exemples reals
 - Apparmor
 - SELinux



Consideracions per aplicar RBAC

- Entendre els requeriments del negoci
 - Per poder determinar els grups d'usuaris
 - Quins permisos ha de tenir cadascun
 - Muntar la infraestructura
- Planificar l'abast de la implantació
 - Quines apps i quins recursos
- Definir els rols
 - Compte amb la manca o l'excès de granularitat
 - Solapament de rols
- Implementació
 - Es recomana començar amb rols que tinguin poca granularitat i anar iterant



Infraestructura per RBAC

- Normalment es defineixen les polítiques a un punt central
 - Active Directory
 - AWS, GCP, Azure
- Es configuren les aplicacions corporatives per fer ús d'aquests rols i grups de privilegis



eXtensible Access Control Markup Language (XACML)

- Una forma habitual de definir el RBAC és a través de XACML
 - És un Standard
 - Permet descriure de forma molt precisa el control d'accés de forma declarativa a través d'un llenguatge
 - Proporciona uns models d'arquitectura i de processament per avaluar les peticions d'accés a través d'unes regles definides a la política
-

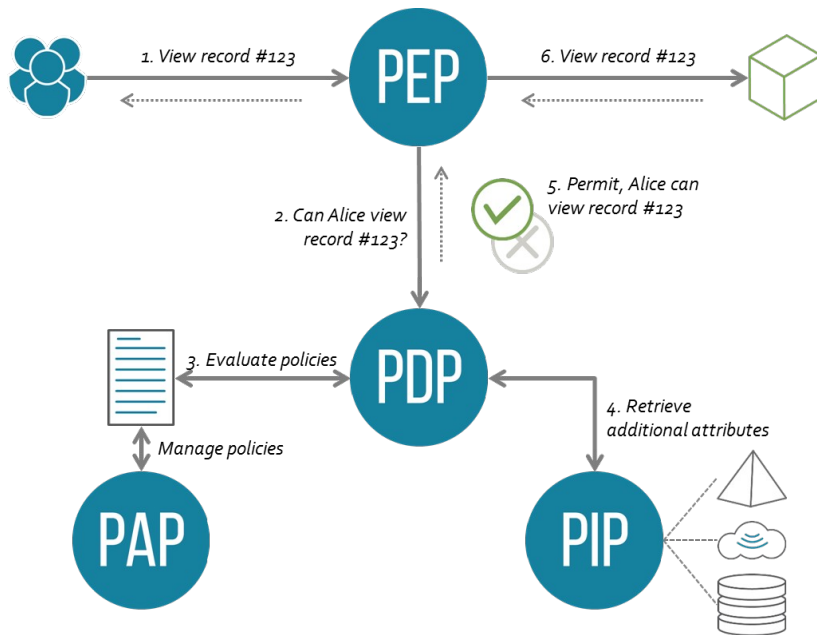


Components de l'Arquitectura de XACML

- **Policy Administration Point (PAP):** Punt de gestió de les polítiques d'autorització
 - **Policy Decision Point (PDP):** Entitat que evalua les peticions d'accés contra les polítiques abans de permetre o denegar una petició
 - **Policy Enforcement Point (PEP):** Punt d'intercepció de les peticions d'un usuari per demanar un recurs. Envia la petició al PDP per obtenir la decisió
 - **Policy Information Point (PIP):** L'entitat que actua com la font dels valors dels atributs, per exemple, el recurs, l'usuari i l'entorn
 - **Policy Retrieval Point (PRP):** Lloc on es guarden les polítiques, pot ser una base de dades o el mateix sistema de fitxers
-



Fluxe de XACML



- Tota petició d'un usuari arriba al Policy Enforcement Point (PEP)
- PEP ho converteix a una petició XACML de petició d'autorització
- S'envia la petició al Policy Decision Point (PDP)
- PDP avalua la petició d'autorització contra les poítiques configurades
 - Aquestes polítiques són adquirides a través del Policy Retrieval Point (PRP) i gestionades pel Policy Administration Point (PAP)
 - Si cal, els atributs s'obtenen del Policy Information Point (PIP)
- El PDP arriba a una decisió que és retornada al PEP
- La decisió pot ser:
 - Permit
 - Deny
 - Not Applicable

RBAC vs ACL

- Des del punt de vista de negoci RBAC > ACL
 - ACL és millor per seguretat a nivell d'usuari individual i amb dades de baix nivell
 - Firewall
 - Sistemes de fitxers...
 - RBAC està pensat per definir polítiques de seguretat a nivell corporatiu
 - Una ACL pot determinar si un usuari pot o no pot escriure un fitxer, però no pot determinar com el pot modificar
-



Inconvenients de RBAC

- Conjunt de regles estàtiques
 - L'administrador les pot canviar però són inherentment estàtiques
- Té el problema de «**role explosion**» a mesura que l'empresa creix o els rols canvien
 - Es van creant nous rols per dotar el sistema de més granularitat
- Això a la pràctica comporta un problema greu d'escalabilitat del sistema



Sistemes de gestió dels drets d'accés d'usuari

- Access Control Lists (ACL)
- Role Based Access Control (RBAC)
- **Attribute Based Access Control (ABAC)**
- Policy Based Access Control (PBAC)



Attribute Based Access Control (ABAC)

- Gestiona els accessos depenent d'uns atributs
 - Contexte o entorn de treball
 - Sistema en qüestió
 - Aplicació usada, ...
- Els permisos es gestionen amb l'avaluació dels valors de certs atributs de forma agregada
- Especialment considera la interrelació dels diferents atributs
- Les regles també es poden escriure amb XACML
- Utilitza parelles clau valor i operacions booleanes
 - `Role=Manager && Category=Financial`



RBAC vs ABAC

- RBAC és molt més estàtic que ABAC ja que els rols estan prefixats
 - Els permisos amb ABAC es donen a través de relacions entre els diferents components
 - Això dóna més granularitat a l'hora d'especificar els permisos
 - Exemple
 - RBAC pot assignar un rol d'administració i tots els qui formin part del rol podran gestionar el sistema
 - Però ABAC permet donar el rol d'administració als usuaris administradors que estan físicament a la oficina
 - El problema és que ABAC requereix més recursos i només convé utilitzar-lo quan RBAC no és suficient
-



Inconvenients d'ABAC

- El principal és la necessitat d'utilitzar XACML amb la complexitat que això suposa a la pràctica
 - Causa vendor lock-in
 - Generalment és complicat trobar experts en aquest llenguatge
- Els managers han de tenir coneixements tècnics per poder adaptar les regles
- Requereix bastant poder computacional quan la quantitat de regles és gran



Sistemes de gestió dels drets d'accés d'usuari

- Access Control Lists (ACL)
- Role Based Access Control (RBAC)
- Attribute Based Access Control (ABAC)
- **Policy Based Access Control (PBAC)**



Policy Based Access Control (PBAC)

- Combina la gestió semàntica de riscos en seguretat amb un sistema dinàmic de polítiques per mitigar amenaces
- És una modificació incremental d'ABAC
 - Suporta també rols i atributs
- Però s'escriu amb llenguatge pla i és molt més senzill
 - Gairebé sempre tenen UI molt senzilles i intuïtives que permeten a qualsevol poder actualitzar la política de seguretat
- Proporciona una interfície automàtica a l'aplicació de MACs

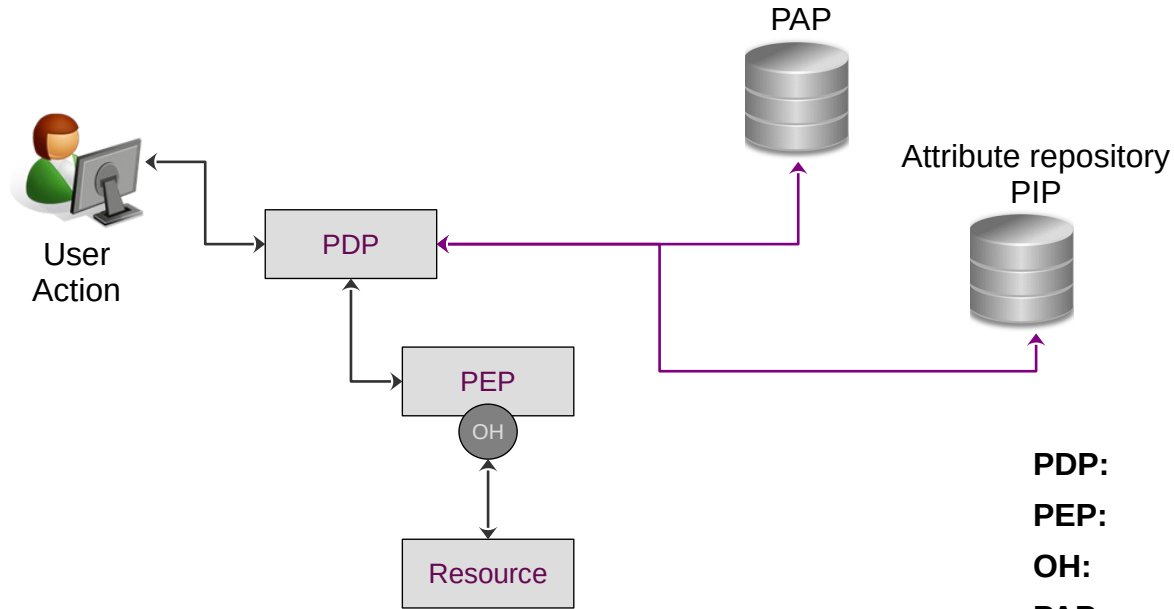


Funcionalitats de PBAC¹

- Flexible amb la tecnologia
 - És agnòstic de l'aplicació consumidora
 - Es proporcionen unes interfícies i el consumidor pregunta pel permís
- Els seus components són flexibles
 - En comptes de confiar amb atributs es confia el procés de decisió a les pròpies dades
- Escalabilitat amb l'empresa
 - El focus és amb escriu la regla un cop fes-la servir molts
 - Exemple: tots els departaments poden accedir a les seves dades
 - La regla és independent del número de departaments ni del número de persones que tenen
- Mapeja molt bé a polítiques de compliment
- Permet treballar en entorns heterogenis
 - Se centralitza la gestió de les polítiques i s'apliquen a tota la corporació

¹ <https://blog.plainid.com/5-key-features-of-policy-based-access-control>

Arquitectura PBAC



PDP: Policy Decision Point
PEP: Policy Enforcement Point
OH: Obligation Handling
PAP: Policy Administration point
PIP: Policy Information Point
 SOAP/WSDL/XACML

Inconvenients de PBAC

- Es continuen necessitant molts recursos
 - Tot i que escala molt millor que ABAC
- La integració no sempre és transparent
 - Cada implementació pot tenir els seus problemes
- No sempre és tant senzill de configurar com sembla



PBAC vs ABAC

- En comptes d'un conjunt d'atributs fem servir les pròpies dades de forma arbitrària per proporcionar el mecanisme d'autorització
 - PBAC no depèn de llenguatges com XACML
 - Podem escriure les regles amb llenguatge natural o a través d'interfícies intuïtives
 - PBAC proporciona millor escalabilitat que ABAC
-



Seguretat en les aplicacions

- Introducció
- Errors més comuns (segons OWASP)
- Gestió de vulnerabilitats
- Auditories
- Sistemes de gestió dels drets d'accés d'usuaris
- **Autenticació i Autorització**
- Sistemes de federació d'identitats: SSO
- Artefactes d'acreditació de drets d'accés (SAML)
- OAuth2

Sistemes d'autenticació vs. autorització

- Autenticació identifica un usuari
- Autorització li dóna permís per accedir a un recurs
- Són dos conceptes complementaris però molt diferents
- Autenticació és el pas previ per poder implementar autorització
 - Fins que no se qui és l'usuari no puc decidir quins privilegis atorgar-li

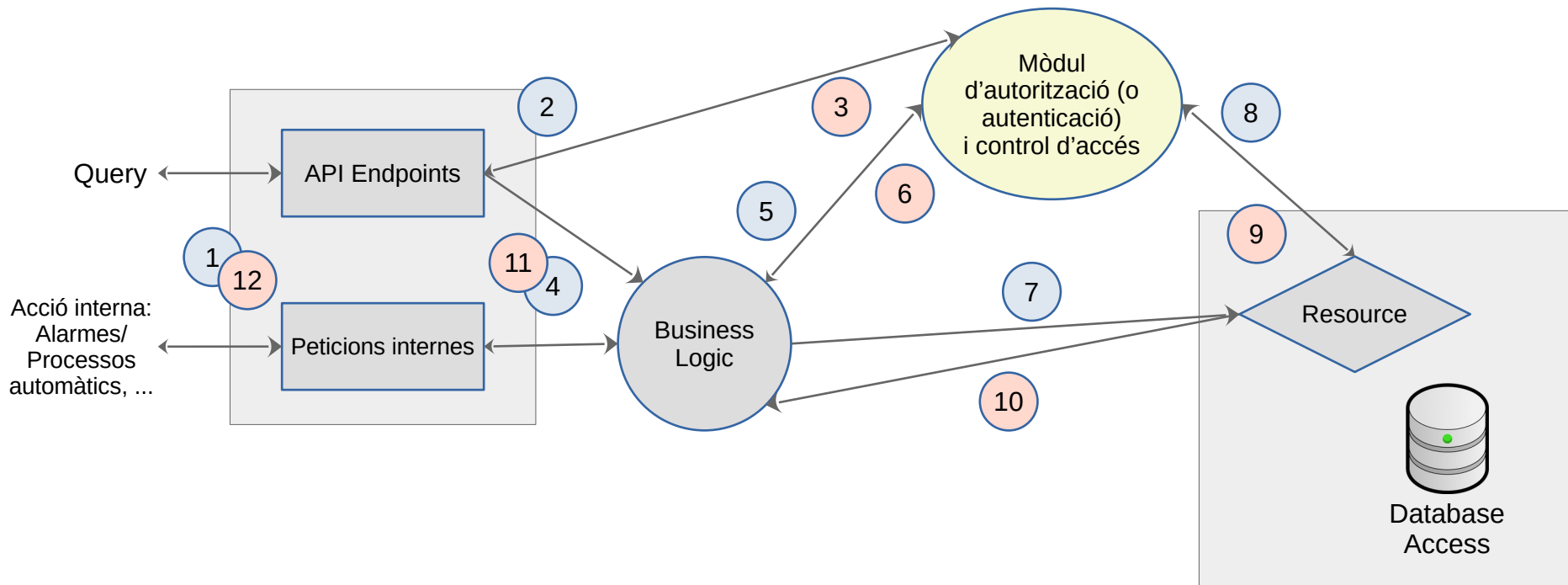


Arquitectura per proporcionar Autenticació i Autorització

- Ambdós convé que es donin a nivell corporatiu
- Tota aplicació i servei ha de preguntar al node responsable
 - Per autenticar-se
 - Per Autoritzar
- Si no és així la petició s'ha de denegar automàticament
 - Principi de mínims privilegis
- Centralitzar és bo ja que segueix el principi de l'únic responsable
 - On cada dada té un responsable i només un
 - Això evita inconsistències i simplifica enormement la gestió



Arquitectura general per Autorització o Autenticació



Privilegis dels usuaris

- Assignar recursos als usuaris és crític
 - Però és encara més important la gestió d'una baixa o bé un canvi en els privilegis assignats a un usuari
 - La forma de procedir vindrà condicionada pel tipus de control d'accés que tinguem configurats
 - És convenient tenir una política clara a l'hora d'assignar privilegis a usuaris
-



Assignació de privilegis – Política d'Accés Mínim

- Cada usuari disposa del mínim conjunt de privilegis que necessita
- Implementar una política de separació de privilegis
- Els privilegis mai s'assignen a l'usuari, s'assignen a nivell de grup
- L'administrador ha de crear el conjunt de grups que satisfacin la política de seguretat de l'empresa
 - De forma equivalent es poden assignar rols
- La política considera qualsevol recurs del sistema
 - Usuaris
 - Documents
 - Equips
 - Serveis, ...



Política d'accés mínim (cont.)

- Mai s'han de realitzar tasques amb un compte privilegiat a no ser que sigui imprescindible
 - Quan hi ha un error el compte privilegiat és molt més disruptiu
 - Quan s'explota una vulnerabilitat, els privilegis de l'atacant seran els del servei vulnerat
- Per poder gestionar efectivament els privilegis s'ha de fer d'un punt centralitzat
 - On es determinin quins nivells de privilegis hi ha
 - Quins recursos es gestionen
 - Quins usuaris tenen quins privilegis



Problemes amb els privilegis

- Hi ha recursos que en moments donats requereixen privilegis d'administrador
 - Instal·lació d'una aplicació local
 - Realització d'una acció concreta per una aplicació, p.e., un backup
- Atorgar aquests privilegis als usuaris normalment significa que se'ls donen per a sempre
- Al cap dels anys s'acaba tenint un entorn amb centenars de recursos amb més privilegis dels necessaris
 - Per negligència de l'administrador
 - Per anar ràpid
 - ...



Avantatges de la política d'accés mínim

- Redueix la superfície d'atac
 - Quan un atacant ha compromès un compte d'usuari pot realitzar menys accions si no té permisos
- Minimitza les possibilitats d'expansió del malware
 - No pot accedir a recursos on l'usuari no hi té privilegis
- Millora la productivitat dels usuaris
 - Se centren amb el treball que poden realitzar
- Simplifica les auditories
 - Ja que obliga a l'administrador a tenir ben organitzats els privilegis dels usuaris



Seguretat en les aplicacions

- Introducció
- Errors més comuns (segons OWASP)
- Gestió de vulnerabilitats
- Auditories
- Sistemes de gestió dels drets d'accés d'usuaris
- Autenticació i Autorització
- **Sistemes de federació d'identitats: SSO**
- Artefactes d'acreditació de drets d'accés (SAML)
- OAuth2

La problemàtica

- Fins fa poc les empreses tenien poques aplicacions que feien moltes funcions
- Però ara hi ha nous models de desenvolupament basats en microserveis i cloud
 - Sovint gestionats per diferents proveïdors (o no)
- El model tradicional d'autenticació i autorització implica constants autenticacions
 - Amb possibles credencials diferents
 - Amb diferents polítiques de caducitat
 - ...



Una mica d'història

- Inicialment es feia servir autenticació amb usuari i contrasenya
- Després es va passar a un model on les xarxes socials podien autenticar-te a serveis externs (login with Google, ...)
- Altres serveis no tenen password
 - A través de magic links o amb One Time Passwords (OTP)
- Autenticació Multi-factor
 - Es requereix un altre dispositiu per verificar la identitat de l'usuari

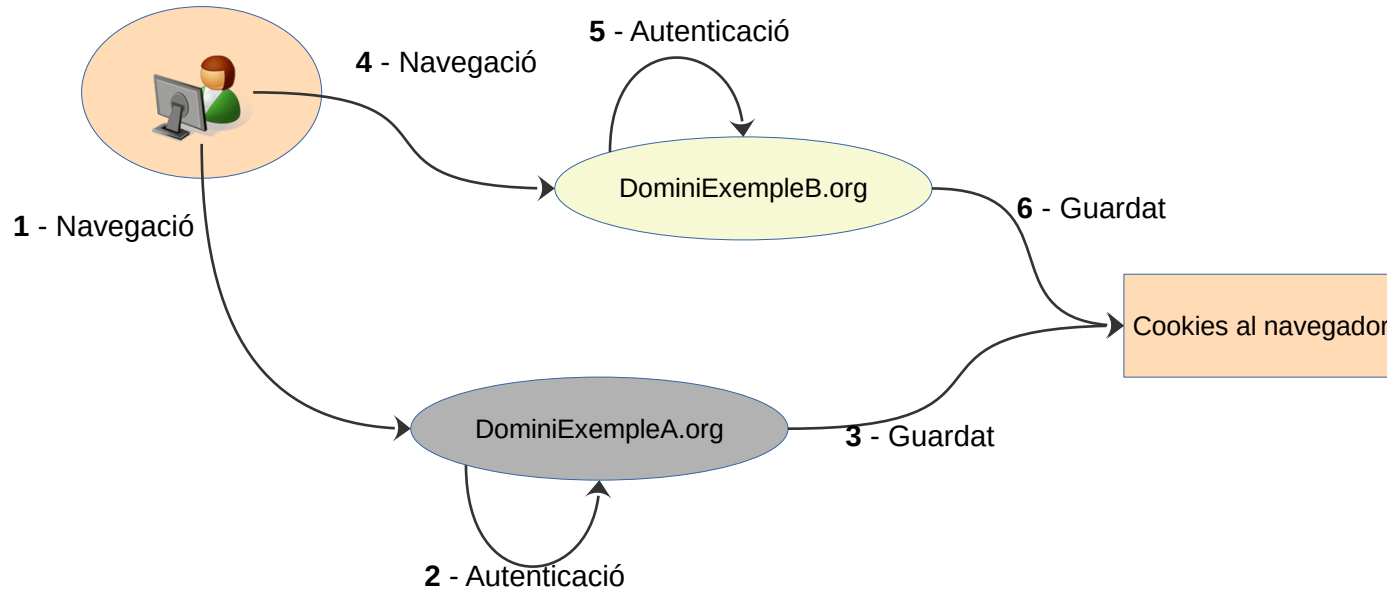


Sistemes de federació d'identitats: SSO

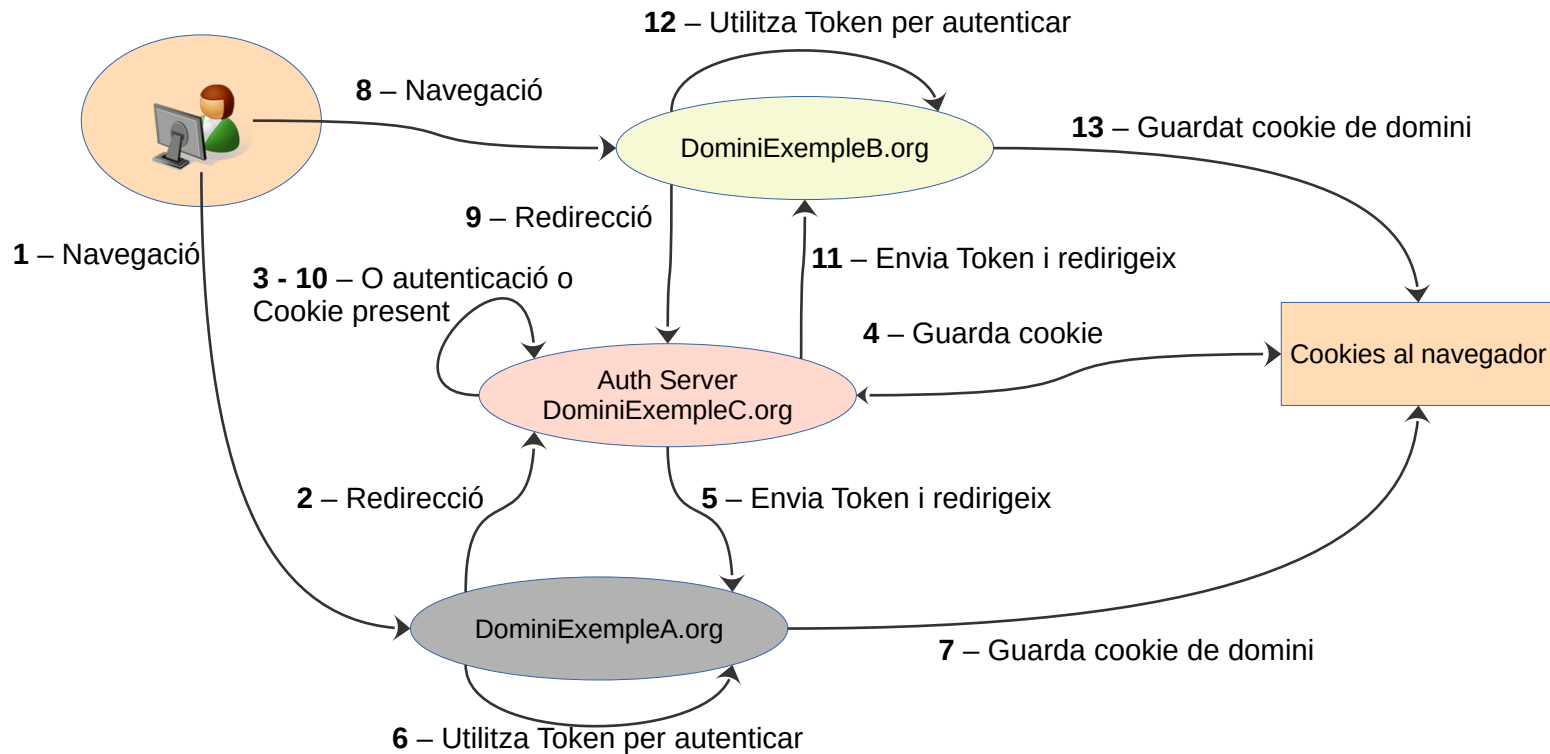
- Tenen per objectiu integrar i simplificar tot el procés d'autenticació
 - L'usuari s'autentica a un sistema centralitzat
 - La resta de serveis validen aquesta autenticació sense requerir més credencials a l'usuari
 - Permetent la simplificació i la integració de tot el workflow de forma transparent
-



Workflow d'autenticació (Sense SSO)



Workflow d'autenticació (Amb SSO)



Solucions per federació d'identitat

- SAML → El veurem en breu
 - OpenID Connect / OAuth → També el veurem de seguida!
 - Proveidors de xarxes socials
 - Lightweight Directory Access Protocol (LDAP)
 - Active Directory (AD)
 - WS-Federation
-



Seguretat en les aplicacions

- Introducció
- Errors més comuns (segons OWASP)
- Gestió de vulnerabilitats
- Auditories
- Sistemes de gestió dels drets d'accés d'usuaris
- Autenticació i Autorització
- Sistemes de federació d'identitats: SSO
- **Artefactes d'acreditació de drets d'accés (SAML)**
- OAuth2

Security Assertion Markup Language (SAML)

- Sistema estandarditzat per «Organization for the Advancement of Structured Information Standards» (OASIS) al 2005
 - Permet transferir informació d'autenticació entre l'Identity Provider i el Service Provider
 - Objectiu de intercanviar informació d'autenticació i d'autorització
 - Es realitza entre l'*identity provider* i el *service provider*
 - Proporciona Assertions
 - Security tokens
 - Informació de l'usuari
 - Usat principalment en entorns basats amb web
-



SAML un compendi d'estàndards

- eXtensible Markup Language (XML): La gran majoria d'intercanvis utilitzen XML
 - XML Schema (XSD): Els diferents components estan definits per XML Schema
 - XML Signature: El xifrat de les dades i les signatures digitals segueixen aquest estàndard
 - XML Encryption: Pels identificadors xifrats, atributs xifrats i assertions xifrades
 - HTTP: Protocol usat per la comunicació
 - Simple Object Access Protocol (SOAP): És el contenidor de transport usat
-



Conceptes de SAML

- **Assertions:** Autenticació, atributs i decisions d'autorització
 - **Protocols:** seqüència de peticions i respostes per obtenir les Assertions i poder realitzar la gestió de la identitat
 - **Bindings:** mapeig de protocols SAML a protocols estàndard de missatgeria i comunicació (SOAP, ...)
 - **Profiles:** Combinació d'accions per poder definir com es realitzen les accions
-



Assertions

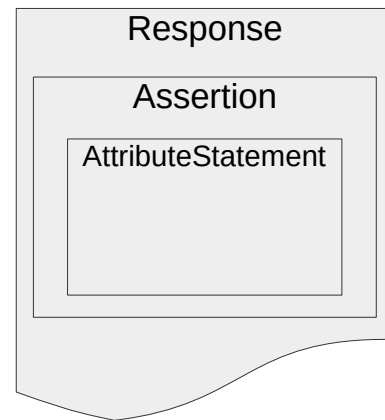
```
<saml:Assertion ...>
  ...
</saml:Assertion>
```

- Conté informació de seguretat
- Contenen instruccions que permeten al proveïdor de serveis prendre decisions de control d'accés
- 3 tipus d'instruccions (statements)
 - D'**autenticació**: indiquen que un principal s'ha autenticat a l'identity provider en un moment donat amb un mecanisme d'autenticació. També es comparteix el contexte d'autenticació
 - D'**atribut**: associa al principal amb uns atributs (parells clau valor)
 - D'**autorització**: indica que el principal pot fer una acció sobre un recurs particular donada una evidència.



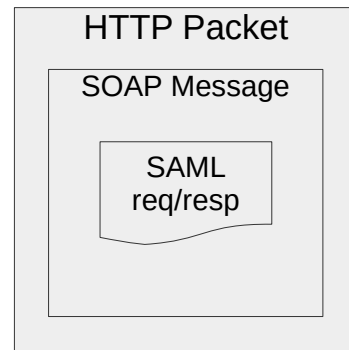
Protocols

- Indiquen com alguns elements s'han d'empaquetar
- Dóna les regles de processament dels elements
- Bàsicament un protocol és un simple request-response
- Hi ha diversos protocols
 - Assertion Query and Request Protocol
 - Authentication Request Protocol
 - Artifact Resolution Protocol
 - Name Identifier Management Protocol
 - Single Logout Protocol
 - Name Identifier Mapping Protocol



Bindings

- Mapeja el protocol a un sistema standard de missatgeria
 - Normalment SAML SOAP
- Defineix la forma en que s'encapsula la informació a transmetre
- Se suporten
 - SAML SOAP Binding (based on SOAP 1.1)
 - Reverse SOAP (PAOS) Binding
 - HTTP Redirect (GET) Binding
 - HTTP POST Binding
 - HTTP Artifact Binding
 - SAML URI Binding

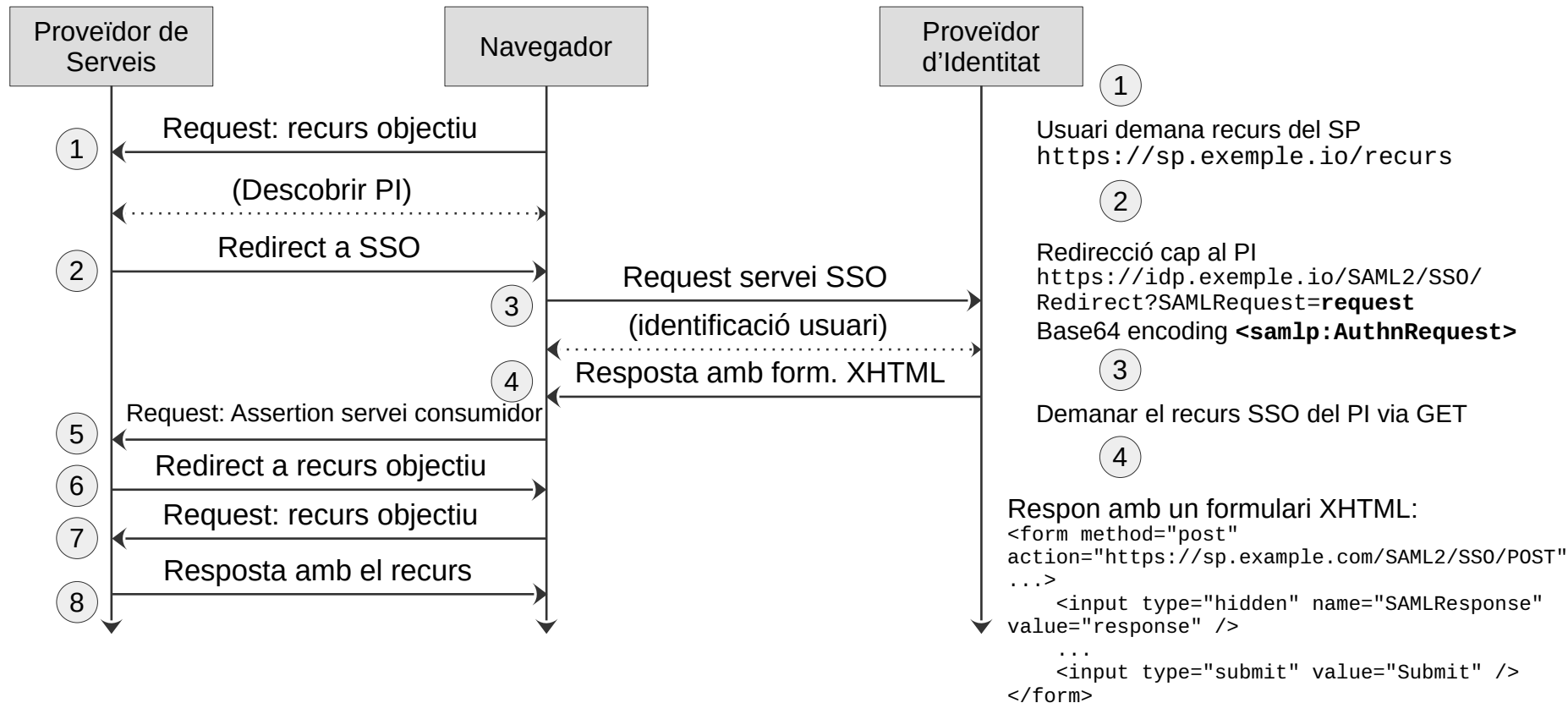


Profiles

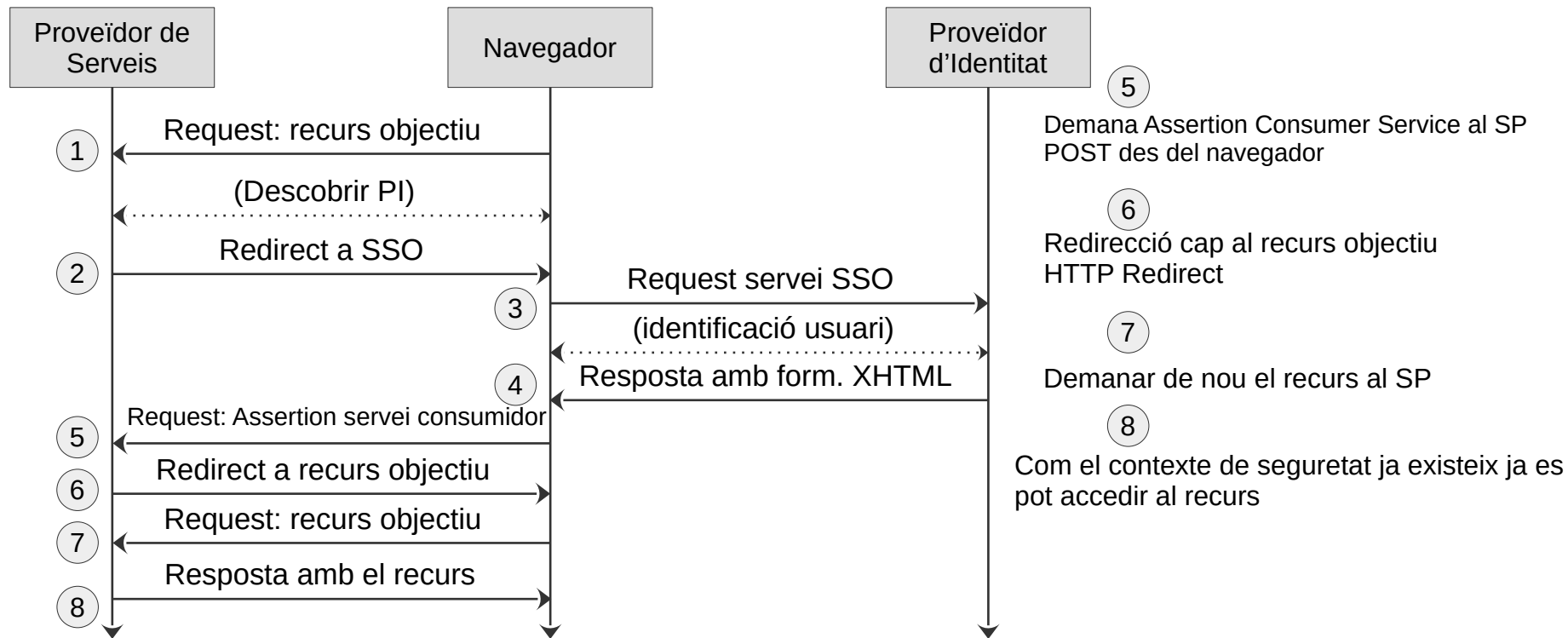
- Defineix la llista d'accions i com formar-les per oferir serveis (casos d'ús)
 - Perfils definits
 - SSO Profiles
 - **Web Browser SSO Profile**
 - Enhanced Client or Proxy (ECP) Profile
 - Identity Provider Discovery Profile
 - Single Logout Profile
 - Name Identifier Management Profile
 - Artifact Resolution Profile
 - Assertion Query/Request Profile
 - Name Identifier Mapping Profile
 - SAML Attribute Profiles
-



Exemple – Web Browser Single Sign-On



Exemple – Web Browser Single Sign-On



Seguretat en les aplicacions

- Mètodes d'identificació dels usuaris
 - Sistemes de gestió dels drets d'accés d'usuari
 - Sistemes d'autenticació vs. autorització
 - Sistemes de federació d'identitats: SSO
 - Artefactes d'acreditació de drets d'accés (SAML)
 - **OAuth2**
 - SOLID
-



Estudi previ: autenticació i autorització WEB

- Algunes coses d'HTTPS
- Autenticació Web típica
 - Autenticació bàsica
 - Autenticació Digest
 - Autenticació basada amb Cookies
 - Autenticació basada amb tokens



Alguns punts sobre HTTP(s)

- HTTP i la seva variant HTTPs són protocols sense estat
 - Per tant cada crida ha de contenir suficient informació de contexte per poder ser interpretada
 - Com a exemple: el login i el password s'envien a cada petició cap al servidor per defecte
 - Per donar una «sensació» d'estat tenim el concepte de sessions
- Un missatge HTTP està compostat per:
 - Capçaleres – Dinàmics i depenent del servei
 - Payload (la pàgina que volem consultar)



Exemple petició HTTPS

The screenshot shows the 'Headers' tab of a web browser's developer tools. The 'General' section displays the following information:

- Request URL: `https://www.google.com/`
- Request Method: `GET`
- Status Code: `200`
- Remote Address: `142.250.184.164:443`
- Referrer Policy: `strict-origin-when-cross-origin`

The 'Response Headers' section displays the following information:

- `accept-ch: Sec-CH-UA-Platform`
- `accept-ch: Sec-CH-UA-Platform-Version`
- `accept-ch: Sec-CH-UA-Full-Version`
- `accept-ch: Sec-CH-UA-Arch`
- `accept-ch: Sec-CH-UA-Model`
- `accept-ch: Sec-CH-UA-Bitness`
- `accept-ch: Sec-CH-UA-Full-Version-List`
- `accept-ch: Sec-CH-UA-WoW64`
- `alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000,h3-0050=":443"; ma=2592000,h3-0046=":443"; ma=2592000,h3-0043=":443"; ma=2592000; v="46,43"`
- `bfcache-opt-in: unload`
- `cache-control: private, max-age=0`
- `content-encoding: br`
- `content-length: 43775`
- `content-type: text/html; charset=UTF-8`
- `date: Thu, 14 Jul 2022 13:28:27 GMT`
- `expires: -1`
- `server: gws`
- `strict-transport-security: max-age=31536000`
- `x-frame-options: SAMEORIGIN`
- `x-xss-protection: 0`

The 'Request Headers' section is partially visible at the bottom, showing a warning icon and the text: 'Provisional headers are shown [Learn more](#)'. The headers listed are:

- `Upgrade-Insecure-Requests: 1`
- `User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.0.0 Safari/537.36`

Model típic d'autenticació Web

- L'usuari es connecta a un sistema on ja hi té un compte
- L'usuari proporciona el ID únic i la clau per verificar la seva identitat
- Les credencials es comparen contra les guardades al servidor
- Un cop l'autenticació valida el compte, es proporciona accés al servei



Mètodes típics d'autenticació web

- Autenticació Bàsica
 - Autenticació per Digest
 - Autenticació basada amb Cookies
 - Autenticació basada amb Tokens
 - Autenticació basada amb One-time Passwords
 - API Key Authentication
-



Autenticació Bàsica

- És la forma més senzilla d'enviament de credencials
- Aquestes no es xifren per defecte, senzillament s'envien codificades amb **base64**
- Això fa imprescindible la utilització de xifrat a nivells inferiors → HTTPS
- Cada petició enviarà el header d'autenticació
 - HTTP és un protocol sense estat



Autenticació Bàsica

```
GET / HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101
Firefox/91.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
DNT: 1
Connection: close
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Authorization: Basic YWRtaW46cGFzc3dvcmQ=
```

→ **Basic**
admin:password



Autenticació Bàsica

Avantatges

- Molt senzill d'implementar
- Suportat nativament per servidors web i navegadors
- Fàcilment scriptable
- No es necessita informació del servidor per poder fer les queries

Inconvenients

- Les credencials viatgen en clar
- Tot i anar xifrat, si es trenqués la seguretat s'exposarien les credencials
- No hi ha la possibilitat de fer logout
- Cada petició envia les credencials
- No es poden expirar les credencials (requereix un canvi de contrasenya)



Autenticació per Digest

- En comptes d'enviar les credencials amb base64 ara les «digerirem»
 - S'envien amb un hash
- Aquest mecanisme requereix més feina a la part del client
- Ha evolucionat des de la seva creació
 - Tot i que diverses extensions no han sigut mai implementades als navegadors
- El major avantatge és que mai s'envia en clar la contrasenya
 - I el contingut depèn de la generació de números aleatoris



Autenticació per Digest

- Es defineixen dues variables a la part del client: HA1 i HA2
- Si el QOP¹ no està definit:
 HA1 = MD5(username:realm:password)
 HA2 = MD5(method:digestURI)
 response = MD5(HA1:nonce:HA2)
- Si el QOP és auth
 HA1 = MD5(MD5(username:realm:password):nonce:cnonce)
 HA2 = MD5(method:digestURI:MD5(entityBody))
 response = MD5(HA1:nonce:nonceCount:cnonce:qop:HA2)

 ¹ Quality of Protection

Autenticació amb Digest

Avantatges

- Eficient en la transmissió
- Suportat nativament per servidors
- Parcialment suportat per navegadors
- Fàcilment scriptable
- Més segur que Basic Authentication

Inconvenients

- Les versions més avançades no estan suportades pels navegadors (SHA256, ...)
- Han aparegut formes més interessants de fer el mateix
- No hi ha la possibilitat de fer logout
- Cada petició envia les credencials
- No es poden expirar les credencials (requereix un canvi de contrasenya)



Autenticació basada amb Cookies

- Utilitza les cookies com a mètode per accedir a la informació de l'usuari
- Permeten donar una molt millor experiència a l'usuari
- És la millor manera d'implementar «sessions»
 - L'estat es guarda a client-side
 - Quan tornem a accedir al contingut es passa la cookie i això ens identifica
 - No fa falta posar la contrasenya cada cop



Què són les (HTTP) cookies?

- Són «fitxers» de text que contenen informació sobre l'usuari
- Estan formats per una llista de parelles clau – valor
 - Contenen informació com ara a quin domini s'ha d'enviar aquesta cookie
 - O qui som
 - O quines pàgines dins del domini hem visitat
- Són enviades pel servidor
 - Només s'envien quan el servidor vol que el client guardi l'estat
- Són enviades pel client
 - Quan torna a la mateixa pàgina i vol recuperar l'estat anterior
 - Carret de la compra
 - Llista de correus
 - ...



Més sobre cookies

- Usos
 - **Gestió de les sessions dels usuaris:** evita haver d'enviar credencials cada cop
 - **Personalització:** anuncis personalitzats, ...
 - **Tracking:** per recordar quins productes ha mirat un usuari, així poder-li fer recomanacions personalitzades
- Tipus de cookies
 - **Cookies de sessió:** es guarden en memòria i serveixen per controlar l'estat de l'usuari durant la seva estada a la pàgina
 - **Cookies persistents:** es guarden a disc, tenen una data d'expiració
 - D'autenticació
 - De tracking



Com funciona la interacció

- 1) El client fa una petició d'autorització
- 2) El servidor valida l'autorització
- 3) El servidor envia la validació junt amb una cookie

Dades presents a la cookie (poden ser diferents):

HTTP/2.0 200 OK

Content-Type: text/html

Set-Cookie: <cookie-name>=<cookie-value>

Set-Cookie: Expires=<date>

Set-Cookie: Max-Age=<number>

Set-Cookie: **Domain**=<domain>

Set-Cookie: Path=<path>

Set-Cookie: **Secure**

Set-Cookie: **HttpOnly**

Seguretat de les Cookies

- El problema és que el client pot canviar les cookies com vulgui
 - I de cops el servidor no guarda aquest estat
 - Les cookies es guarden a disc en clar, si hi ha credencials, ...
 - Els ID de sessió també poden ser modificats, però ens ajuden a guardar estat a server-side
- Tot i així hi ha cookies que ens poden ajudar fent el sistema més segur
 - Només s'envien quan hi ha canvis, es minimitza la exposició a la xarxa
 - La clau **Secure** obliga l'ús d'HTTPS o el navegador atura la transferència
 - **HttpOnly** fa que Javascript no pugui llegir les cookies, pel que evita atacs de scripting
- Què podem fer per evitar les modificacions de les cookies?



Seguretat en les cookies: Evitar modificacions

- Cal tenir clar que no podem evitar la modificació de les cookies
- PERÒ si podem detectar els canvis, o fer que no es puguin llegir pel client
 - Això s'aconsegueix amb les Cookies signades
- Bàsicament les cookies es guarden a una «cookie jar»
 - Ara tindrem dues cookie jars: la signada i la no signada



Bones pràctiques amb cookies

- Requerir una connexió segura
 - Com? Cookie Secure
- Restringir la cookie a l'aplicació que afecta
 - No a tot el domini
 - Com?:
 - Path i Domain cookies
- Restringir l'accés a les cookies a Javascript
 - Com?
 - HttpOnly



Autenticació basada amb Tokens

- Les cookies un dels problemes que tenen és que guarden més informació de la necessària
 - Els token intenten mitigar aquest problema
 - En comptes de tenir diversos valors ara el que farem serà generar un token i passar-lo com un header d'autenticació
 - La forma més habitual és JWT (JSON Web Tokens)
 - Però hi ha altres standards, p.e. OpenID
-



JSON Web Tokens

- Workflow:
 - 1) El client envia una petició d'autenticació
 - 2) Es verifiquen les credencials i es retorna un token (JSON) signat amb una clau secreta
 - 3) La resta de les interaccions amb el servidor portaran el token:

Authorization: Bearer <token>

- Els token són JSON:

Header + '.' +	Payload	+ '.' + Signature
<pre>{ "alg": "HS256", "typ": "JWT" }</pre>	<pre>{ "sub": "1234567890", "name": "Joan Ribot", "admin": true }</pre>	<pre>HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)</pre>

Sobre tokens d'autenticació

Avantatges

- Tenen més flexibilitat
- Estan pensats per autenticació
- Poden anar xifrats més fàcilment
- Disposen de més «Storages» que les cookies
- Poden tenir temps d'expiració molt petits (5 minuts) → Més seguretat
 - Això es pot fer amb dos tokens: Authentication token i Refresh token

Inconvenients

- Implementació manual
- Com s'ha de fer al client → Javascript
- Workflow més complicat
- Poden ser vulnerable a atacs XSS ja que estan guardats als clients



Autenticació basada amb One-time Passwords (OTP)

- Útils per evitar configurar i tenir un password guardat a un servidor
- Intentar utilitzar contrasenyes més segures del que un humà podria fer
- La idea és generar un password d'un sol ús
 - Per exemple com part de multi-factor authentication
 - O com a magic link després del registre d'un nou usuari
 - O amb la utilització dels Authenticators
 - Generat quan ens autèntiquem amb una smartcard
- En tot cas el password es genera automàticament i se li dóna de forma transparent a l'usuari



Més sobre OTP

- Pot variar entre implementacions però en general:
 - Es genera un Hashed Message Authentication Code (HMAC)
 - Amb alguna variable temporal, per exemple un Timestamp
 - O bé un comptador, per exemple d'events
- Un cop generat es transmet al client a través de:
 - L'autenticator APP
 - SMS → Comença a estar «deprecated»
 - Mail
 - Una aplicació dedicada



Avantatges dels OTP

- Evita la gran majoria de problemes de seguretat dels passwords
 - Curts
 - Endevinables, ...
- Evita haver de compartir contrasenyes
- Evita la repetició de contrasenyes a diversos serveis
- Caduquen molt ràpid



API Key Authentication

- Aquest mètode és relativament similar al OTP
 - En aquest cas és fa servir entre aplicacions que utilitzen HTTP
 - La clau és utilitzada per desenvolupadors i integrada a les apps
 - S'utilitza la clau a cada request (que va xifrada)
 - Normalment és genera de forma automàtica
 - Té un mínim de 30bytes aleatoris
-



Sobre OAuth

- Open Authorization (que no authentication)
- És un altre mecanisme per poder realitzar SSO
 - Autenticar-se un sol cop per serveis que no tenen res a veure entre si
- Es pot fer a través de qualsevol forma d'autenticació
 - OAuth és agnòstic de l'autenticació
- És un standard obert
 - Que permet accedir a usuaris autenticats a un lloc a un altre sense problemes
- Es considera una autorització delegada segura d'un agent d'usuari (navegador) o d'un tercer



Història d'OAuth

- Inicialment suportat per Twitter i Google principalment
- La versió 1 esdevé un standard al 2010
 - És un protocol
- S'adopta massivament per les empreses que el suporten i per la seva simplicitat
- A 2012 es fa la versió 2.0
 - Ara es converteix amb un Framework
 - Va ser molt criticada per ser menys segura en segons quins aspectes a la v1.0
- Ara el suport és majoritari: Amazon, Facebook, Instagram, LinkedIn, Microsoft, Netflix, Paypal



Conceptes OAuth¹

- Roles OAuth
- Secrets
- Tipus de Grants
- Scopes
- Tokens
- Authentication Workflow

¹ <https://oauth.net/>

Rols OAuth

- **Authorization server** – Servidor a càrrec de l'autenticació. És coneix molts cops com a Identity Provider IdP
 - Gestiona la informació de l'end-user, el seu accés i les relacions de confiança
 - Genera els tokens de seguretat per les APPs i APIs
 - Dóna, limita o revoca els permisos (autorització) per als usuaris del sistema (autenticació)
 - **Client** – Indica l'aplicació que demana accés al recurs protegit
 - Pot ser una el servidor d'una Web App o l'App corrent al navegador
 - En un entorn d'aplicacions pot ser la API que crida a alguna altra API
 - **Resource owner** – El propietari del recurs: l'usuari (end-user)
 - Qui accedeix a les dades és l'aplicació
 - **Resource server** – dóna accés a les dades requerides
 - Generalment implementat com una API gestionant l'accés a les dades
 - Necessita de l'«**Authorization server**» per determinar si l'end-user està autoritzat a accedir al recurs
-



Secrets

- OAuth contempla l'ús de secrets per tal de configurar i permetre l'accés al sistema
- Per evitar tenir problemes de privacitat es genera un Client ID i un Secret que no són els de l'usuari
- Client ID
 - Identifica la app → Pel que en tindrem un per aplicació
- Secret
 - Va aparellat amb el Client ID
 - Convé mantenir-lo secret - això és especialment important sabent que el tindrem dins d'una aplicació



Tipus de Grants

- **Authorization Code Grant** – El servidor retorna un Authorization Code d'un sol us
 - El fem servir per aconseguir l'Access Token
 - És el comportament habitual de web apps
 - Compte amb el Secret! → millor usar PKCE
- **Implicit Grant** – Opció «deprecated» per aconseguir directament l'Access Token
 - La pròpia URI de callback conté el Token
- **Authorization Code Grant with Proof Key for Code Exchange (PKCE)** – Equivalent al Authorization Code grant però amb un pas extra per assegurar les WebApps i les Mobile Apps.
- **Resource Owner Credentials Grant Type** – Model limitat als clients de confiança, qui són els propietaris del recurs
 - En aquest cas no hi ha redirecció a l'Authorization server
- **Client Credentials Grant Type** – utilitzat per aplicacions que treballen de forma automatitzada
 - S'autentica directament amb el Client ID i el secret
- **Device Authorization Flow** – fluxe utilitzat per dispositius on és difícil escriure-hi. p.e. SmartTV
- **Refresh Token Grant** – El fluxe proporciona un Access Token nou a través d'un Refresh Token (JWT)



Scopes

- Indiquen el motiu pel que es demana l'Autorització
- OAuth no defineix quins poden ser
- Depèn del Resource Server
 - L'App declara què vol fer
 - Depenent del què el resource server li dóna permís per fer-ho o no



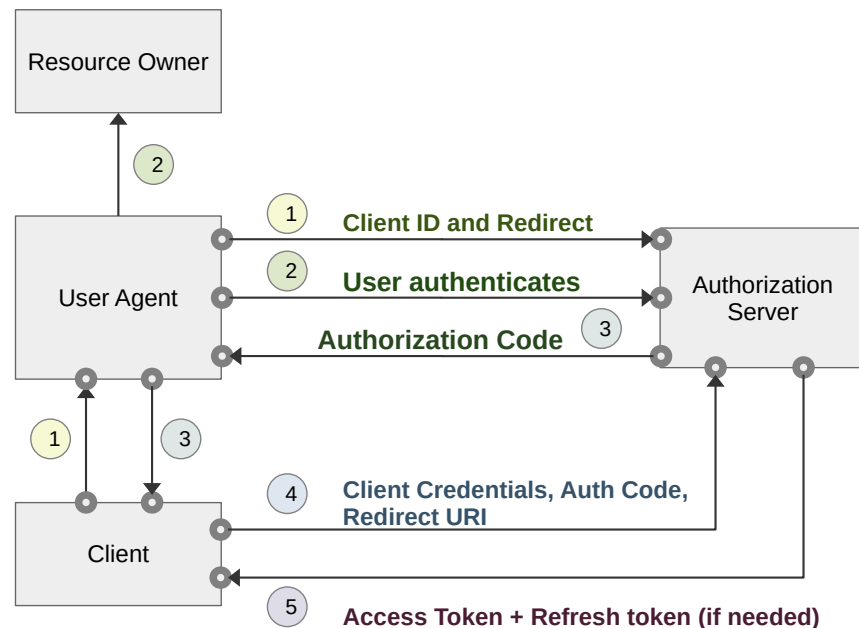
Tokens

- No són més que el codi que obtenim per poder utilitzar el servei
- S'han de guardar amb cura a la part del client
- N'hi ha principalment 3 tipus
 - **Access Token** – Token que dóna accés a un recurs
 - Donat per l'Authorization Server al client
 - **Refresh Token** – Token que ens permet refrescar l'Access Token quan caduca
 - Donat generalment al mateix moment que l'Access Token
 - **Authorization Code** – Codi temporal
 - Incrementa la seguretat fent que no sigui necessari que el navegador vegi l'Access Token



Authentication Workflows¹ – Authorization Code Flow

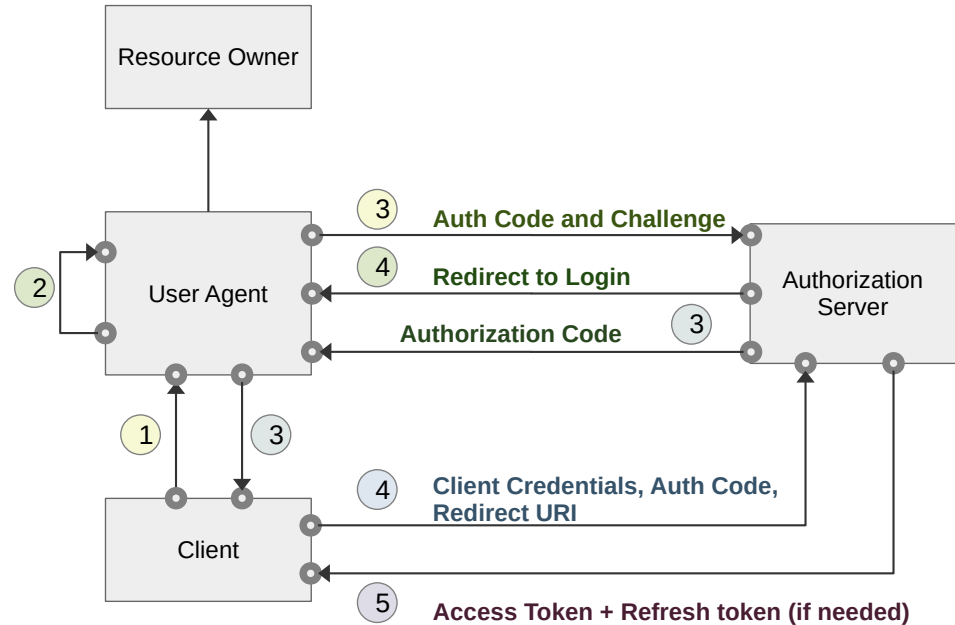
- 1 **Envia el client identifier, requested scope, local state, and a redirection URI**
- 2 **Autentica el resource owner i es denega o accepta l'accés**
- 3 **Si es permet l'accés s'envia l'Auth code i es redirigeix el Client cap a la URI (Auth Server)**
- 4 **S'envien les credencials i el Auth Code i la URI**
- 5 **Si tot va bé es rep el Access Token i el Refresh Token**



¹ <https://www.ibm.com/docs/en/tfim/6.2.2.6?topic=overview-oauth-20-workflow>

Authorization Code Flow with PKCE

- 1 **Es fa el login**
Es genera el Code Verifier i el Code Challenge
- 2 **Authorization code request + Code Challenge to authorize**
- 3 **Redirect to login and authentication**
- 4 **Si tot va bé es rep el Access Token i el Refresh Token**



Seguretat en les aplicacions

- Mètodes d'identificació dels usuaris
 - Sistemes de gestió dels drets d'accés d'usuari
 - Sistemes d'autenticació vs. autorització
 - Sistemes de federació d'identitats: SSO
 - Artefactes d'acreditació de drets d'accés (SAML)
 - OAuth2
 - **SOLID** → Fork de transparències de l'Albert Renom)
-



Disseny Segur

- Tenir un codi segur també passa per tenir un codi que tingui pocs bugs, i sigui fàcil de mantenir.
- Per aconseguir això cal que nostre codi apliqui el disseny SOLID de desenvolupament de software. Per contra quan el nostre codi es complex de mantenir, i està farcit de bugs diem que en el nostre codi és STUPID



Disseny Segur: què no s'ha de fer

- **STUPID** és un acrònim on cada lletra vol dir algun error de disseny que hi ha en el nostre codi:
 - **S**: Singelton
 - **T**: Tight Coupling (acoblament). El nostre codi és molt poc tolerant a canvis
 - **U**: Untestability (codi que no es pot testejar)
 - **P**: Premature optimization (codi no optimitzat de forma eficient)
 - **I**: Indescriptive Naiming (només de funcions, variables i mètodes que no ens indiquen res, cal recordar que escrivim codi per a les persones no per les màquines)
 - **D**: Duplication (codi duplicat)
-



Disseny Segur: què hem de fer

- **SOLID** és un acrònim on cadascuna de les lletres representa un principi de Disseny
 - **S** : Single Responsibility principle (SRP)
 - **O**: Open/Closed principle (OCP)
 - **L**: Liskov Substitution Principle (LSP)
 - **I**: Interface segregation Principle (ISP)
 - **D**: Dependency Inversion Principle (DIP)
-



Disseny Segur: SRP

- Aquest principi ens ve a dir que cada classe hauria de tenir només una responsabilitat

```
public class UserService
{
    public void Register(string email, string password)
    {
        if (!ValidateEmail(email))
            throw new ValidationException("Adreça de correu invàlida");
        var user = new User(email, password);

        SendEmail(new MailMessage("mysite@nowhere.com", email) {
            Subject="Benvingut" }); }
    public virtual bool ValidateEmail(string email)
    {
        return email.Contains("@");
    }
    public bool SendEmail(MailMessage message)
    {
        _smtpClient.Send(message);
    }
}
```



Disseny Segur: SRP

```
public class UserService
{
    EmailService _emailService;
    DbContext _dbContext;
    public UserService(EmailService aEmailService, DbContext aDbContext)
    {
        _emailService = aEmailService;
        _dbContext = aDbContext;
    }
    public void Register(string email, string password)
    {
        if (!_emailService.ValidateEmail(email))
            throw new ValidationException("Email is not an email");
        var user = new User(email, password);
        _dbContext.Save(user);
        emailService.SendEmail(
            new MailMessage("myname@mydomain.com", email){
                Subject="Benvingut!"
            }
        );
    }
}
```

```
public class EmailService
{
    SmtpClient _smtpClient;
    public EmailService(SmtpClient aSmtpClient)
    {
        _smtpClient = aSmtpClient;
    }
    public bool virtual ValidateEmail(string email)
    {
        return email.Contains("@");
    }
    public bool SendEmail(MailMessage message)
    {
        _smtpClient.Send(message);
    }
}
```



Disseny Segur: SRP

- D'aquesta manera, hem delegat responsabilitats específiques a classes separades de manera que cada classe només té una raó de canvi. Això augmenta la cohesió i el desacoblament.
- El SPR a més fa que la lògica de negoci d'alt nivell ha d'evitar tractar amb la implementació de baix nivell.



Disseny Segur: OCP

- Aquí, "Obert per extensió" significa que hem de dissenyar el nostre mòdul/classe de manera que la nova funcionalitat només pugui ser afegida quan es generin nous requisits.
- "Tancat per modificació" significa que ja hem desenvolupat una classe i ha passat per proves unitàries. No hauríem de modificar-la fins que trobem errors.
- Amb C#, Open/Closed principle pot ser tractat de les formes següents:
 - Utilitzant Function Parameters
 - Utilitzant Extension methods
 - Utilitzant Classes, Abstract class, o Interface-based Inheritance
 - Generics
 - Composition and Dependency Injection



Disseny Segur: OCP

- Suposem que tenim una classe Rectangle amb les propietats Alçada i Amplada

```
public class Rectangle{  
    public double Height {get;set;}  
    public double Wight {get;set; }  
}
```

- La nostra aplicació ha de calcular la superfície total d'una collecció de rectangles. Com ja hem après el SRP), no necessitem posar el codi de càlcul de la superfície total dins del rectangle. Així que aquí, he creat una altra classe per al càlcul de la superfície.

```
public class AreaCalculator {  
    public double TotalArea(Rectangle[] arrRectangles)  
    {  
        double area;  
        foreach(var objRectangle in arrRectangles)  
        {  
            area += objRectangle.Height * objRectangle.Width;  
        }  
        return area;  
    }  
}
```

Disseny Segur: OCP

```
public class Rectangle{  
    public double Height {get;set;}  
    public double Wight {get;set; }  
}
```

```
public class Circle{  
    public double Radius {get;set;}  
}
```

```
public class AreaCalculator  
{  
    public double TotalArea(object[] arrObjects)  
    {  
        double area = 0;  
        Rectangle objRectangle;  
        Circle objCircle;  
        foreach(var obj in arrObjects)  
        {  
            if(obj is Rectangle)  
            {  
                area += obj.Height * obj.Width;  
            }  
            else  
            {  
                objCircle = (Circle)obj;  
                area += objCircle.Radius * objCircle.Radius * Math.PI;  
            }  
        }  
        return area;  
    }  
}
```



Disseny Segur: OCP

```
public class Rectangle: Shape
{
    public double Height {get;set;}
    public double Width {get;set;}
    public override double Area()
    {
        return Height * Width;
    }
}

public class Circle: Shape
{
    public double Radius {get;set;}
    public override double Area()
    {
        return Radius * Radius * Math.PI;
    }
}
```

```
public abstract class Shape
{
    public abstract double Area();
}

public class AreaCalculator
{
    public double TotalArea(Shape[] arrShapes)
    {
        double area=0;
        foreach(var objShape in arrShapes)
        {
            area += objShape.Area();
        }
        return area;
    }
}
```



Disseny Segur: OCP

- Ara el nostre codi segueix SRP i OCP al mateix temps. Sempre que introduïu una nova forma derivant de la classe abstracta "Shape", no cal que canvieu la classe "AreaCalculator".
 - Avantatges OCP:
 - 1) Minimitza les possibilitats d'error mitjançant la no modificació de les classes existents.
 - 2) Permet afegir noves funcionalitats amb facilitat afegint noves classes, sense que les funcionalitats actuals depenguin de les noves classes.
 - 3) Promou el Principi de la Responsabilitat Única (SRP), ja que cada classe es centra en una única responsabilitat.
 - 4) Facilita la prova unitària de cada classe, ja que les classes es poden provar de manera independent.
-



Disseny Segur: LSP

- El Principi de Substitució de Liskov (LSP) guia com utilitzar la herència en la programació orientada a objectes. Es tracta de la subtipificació i com derivar correctament un tipus a partir d'un tipus base.

Els subtipus han de ser substituïbles pel seu tipus base.

- Simplificant-ho encara més, una classe derivada ha de ser correctament substituïble per la seva classe base. Quan derives una classe a partir d'una classe base, la classe derivada ha de implementar correctament tots els mètodes de la classe base. No ha de suprimir alguns mètodes llançant NotImplementedException.
-



Disseny Segur: LSP

- Suposem que necessitem crear una aplicació per gestionar dades mitjançant un grup de fitxers de text SQL. Aquí necessitem escriure funcionalitats per carregar i desar el text d'un grup de fitxers SQL al directori de l'aplicació. Així que necessitem una classe que gestioni la càrrega i mantingui el text d'un grup de fitxers SQL juntament amb la classe `SqlFile`

```
public class SqlFile
{
    public string FilePath {get;set;}
    public string FileText {get;set;}
    public string LoadText()
    {
        /* Code to read text from sql file */
    }
    public string SaveText()
    {
        /* Code to save text into sql file */
    }
}

public class SqlFileManager
{
    public List<SqlFile> lstSqlFiles {get;set}

    public string GetTextFromFiles()
    {
        StringBuilder objStrBuilder = new StringBuilder();
        foreach(var objFile in lstSqlFiles)
        {
            objStrBuilder.Append(objFile.LoadText());
        }
        return objStrBuilder.ToString();
    }
    public void SaveTextIntoFiles()
    {
        foreach(var objFile in lstSqlFiles)
        {
            objFile.SaveText();
        }
    }
}
```

Disseny Segur: LSP

- Després d'un temps, els nostres caps podrien dir-nos que podem tenir alguns fitxers de només lectura al directori de l'aplicació, de manera que hem de restringir el flux quan intenti desar-los.

```
public class SqlFileManager
{
    public List<SqlFile> lstSqlFiles {get;set}
    public string GetTextFromFiles()
    {
        StringBuilder objStrBuilder = new StringBuilder();
        foreach(var objFile in lstSqlFiles)
        {
            objStrBuilder.Append(objFile.LoadText());
        }
        return objStrBuilder.ToString();
    }
    public void SaveTextIntoFiles()
    {
        foreach(var objFile in lstSqlFiles)
        {
            if(! objFile is ReadOnlySqlFile)
                objFile.SaveText();
        }
    }
}
```



Disseny Segur: LSP

```
public interface IReadableSqlFile
{
    string LoadText();
}
public interface IWritableSqlFile
{
    void SaveText();
}
public class SqlFile: IWritableSqlFile,IReadableSqlFile
{
    public string FilePath {get;set;}
    public string FileText {get;set;}
    public string LoadText()
    {
        /* Code to read text from sql file */
    }
    public void SaveText()
    {
        /* Code to save text into sql file */
    }
}
```

```
public class SqlFileManager
{
    public string GetTextFromFiles(List<IReadableSqlFile> aLstReadableFiles)
    {
        StringBuilder objStrBuilder = new StringBuilder();
        foreach(var objFile in aLstReadableFiles)
        {
            objStrBuilder.Append(objFile.LoadText());
        }
        return objStrBuilder.ToString();
    }
    public void SaveTextIntoFiles(List<IWritableSqlFile> aLstWritableFiles)
    {
        foreach(var objFile in aLstWritableFiles)
        {
            objFile.SaveText();
        }
    }
}
```

Disseny Segur: ISP

- El Principi de Segregació d'Interfícies estableix que:
 - *Els clients no haurien de ser forçats a implementar interfícies que no utilitzen. En lloc d'una única interfície completa, es prefereixen moltes interfícies petites basades en grups de mètodes, cadascuna servint a un submòdul*
- Ho podem definir d'una altra manera. Una interfície hauria d'estar més relacionada amb el codi que la utilitza que amb el codi que la implementa. Així, els mètodes de la interfície es defineixen segons els mètodes que necessita el codi client, en comptes dels mètodes que implementa la classe. Per tant, els clients no haurien de dependre d'interfícies que no utilitzen



Disseny Segur: ISP

- Comencem amb un exemple que incompleix l'ISP. Suposem que necessitem crear un sistema per a una empresa de TI que conté rols com a TeamLead i Programmer, on TeamLead divideix una tasca gran en tasques més petites i les assigna als seus programadors o pot treballar directament en elles

```
public Interface ILead
{
    void CreateSubTask();
    void AssignTask();
    void WorkOnTask();
}
public class TeamLead : ILead
{
    public void AssignTask()
    {
        //Code to assign a task.
    }
    public void CreateSubTask()
    {
        //Code to create a sub task
    }
    public void WorkOnTask()
    {
        //Code to implement perform assigned task.
    }
}
```



Disseny Segur: ISP

- El disseny sembla correcte per ara. No obstant això, més endavant s'introdueix un altre rol al sistema, com ara el **Manager**, que assigna tasques al TeamLead i no treballa en elles. Podem implementar directament una interfície ILead a la classe Manager, com es mostra a continuació?

```
public class Manager: ILead
{
    public void AssignTask()
    {
        //Code to assign a task.
    }
    public void CreateSubTask()
    {
        //Code to create a sub task.
    }
    public void WorkOnTask()
    {
        throw new Exception("Manager can't work on Task");
    }
}
```



Disseny Segur: ISP

- Tenim 3 rols:
 - 1) Els gestors només poden dividir i assignar tasques
 - 2) Els TeamLead pot dividir i assignar les tasques i treballar-hi,
 - 3) Els programadors que només pot treballar en les tasques



Disseny Segur: ISP

```
public class TeamLead: IProgrammer, ILead
{
    public void AssignTask()
    {
        //Code to assign a Task
    }
    public void CreateSubTask()
    {
        //Code to create a sub task from a task.
    }
    public void WorkOnTask()
    {
        //code to implement to work on the Task.
    }
}

public interface ILead
{
    void AssignTask();
    void CreateSubTask();
}
```

```
public interface IProgrammer
{
    void WorkOnTask();
}

public class Programmer: IProgrammer
{
    public void WorkOnTask()
    {
        //code to implement to work on the Task.
    }
}

public class Manager: ILead
{
    public void AssignTask()
    {
        //Code to assign a Task
    }
    public void CreateSubTask()
    {
        //Code to create a sub task from a task.
    }
}
```

Disseny Segur: ISP

- En aquest cas, hem separat les responsabilitats, els hem distribuït en múltiples interfícies i hem proporcionat una bona abstracció.
 - El Principi de Segregació d'Interfícies (ISP) no es limita només a les interfícies, sinó que es pot utilitzar amb classes abstractes o qualsevol classe que proporcioni serveis al codi del client.
 - L'ISP ajuda a implementar el Principi de Substitució de Liskov, augmenta la cohesió i, en conseqüència, dóna suport al Principi de Responsabilitat Única.
-



Disseny Segur: ISP

Els següents indicis de codi detecten una violació de l'ISP:

- Quan teniu interfícies grans.
- Quan implementeu una interfície en una classe concreta en què alguns mètodes no tenen cap codi d'implementació o llancen `NotImplementedException`.
- Quan només crides a un petit conjunt de mètodes d'una interfície més gran.

La solució per a les violacions de l'ISP seria:

- Dividir les interfícies grans en altres de més petites.
- Heretar múltiples interfícies petites si és necessari.
- Utilitzar el patró de disseny de l'adaptador per a interfícies grans de tercers, de manera que el vostre codi pugui funcionar amb l'adaptador.



Disseny Segur: DIP

- El Principi d'Inversió de la Dependència estableix que:
 - Els mòduls d'alt nivell no han de dependre dels mòduls de baix nivell. Ambdós han de dependre de l'abstracció.
 - Un mòdul de nivell alt és un mòdul (classe) que utilitza altres mòduls (classes) per realitzar una tasca.
 - Un mòdul de nivell baix conté una implementació detallada d'alguna tasca específica que poden utilitzar altres mòduls
-



Disseny Segur: DIP

```
public class Student
{
    public int StudentId { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public DateTime DoB { get; set; }

    //tight coupling
    private StudentRepository _stdRepo =
        new StudentRepository();

    public Student()
    {
    }

    public void Save()
    {
        _stdRepo.AddStudent(this);
    }
}
```

```
public class StudentRepository
{
    public void AddStudent(Student std)
    {
        //EF code removed for clarity
    }

    public void DeleteStudent(Student std)
    {
        //EF code removed for clarity
    }

    public void EditStudent(Student std)
    {
        //EF code removed for clarity
    }

    public IList<Student> GetAllStudents()
    {
        //EF code removed for clarity
    }
}
```

Disseny Segur: DIP

El problema és que la classe `Student` crea un objecte de la classe `StudentRepository` concreta utilitzant la paraula clau `new` i això fa que estiguin fortament acoblades. Això comporta els següents problemes:

- La creació d'objectes utilitzant la paraula clau `new` en tots els llocs és un codi repetit. La creació d'objectes no està en un sol lloc. Violació del principi No Repeteixis Codi (DRY, per les seves sigles en anglès). Si hi ha algun canvi en el constructor de la classe `StudentRepository`, llavors cal fer els canvis en tots els llocs. Si la creació d'objectes és en un sol lloc, seria més fàcil mantenir el codi.
 - La creació d'un objecte utilitzant `new` també fa impossible la realització de proves unitàries. No podem fer proves unitàries de la classe `Student` de manera independent.
 - La classe `StudentRepository` és una classe concreta, de manera que qualsevol canvi en la classe requerirà canvis també a la classe `Student`.
-



Disseny Segur: DIP

```
public class Student
{
    public int StudentId { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public DateTime DoB { get; set; }

    private IStudentRepository _stdRepo;

    public Student(IStudentRepository stdRepo)
    {
        _stdRepo = stdRepo;
    }
    public void Save()
    {
        _stdRepo.AddStudent(this);
    }
}

public interface IStudentRepository
{
    void AddStudent(Student std);
    void EditStudent(Student std);
    void DeleteStudent(Student std);

    IList<Student> GetAllStudents();
}
```

```
public class StudentRepository : IStudentRepository
{
    public void AddStudent(Student std)
    {
        //code removed for clarity
    }
    public void DeleteStudent(Student std)
    {
        //code removed for clarity
    }
    public void EditStudent(Student std)
    {
        //code removed for clarity
    }
    public IList<Student> GetAllStudents()
    {
        //code removed for clarity
    }
}
```



Disseny Segur: DIP

- Això redueix el nivell d'acoblament i fa que cada classe sigui testeable de forma independent.
- Qui faci la crida a la classe Student pot passar un objecte de qualsevol classe que implementi la interfície IStudentRepository, i així no queda lligat a una classe concreta específica
- Aquesta flexibilitat és beneficiosa perquè permet la substitució de classes en temps d'execució, el que pot ser útil en diverses situacions, com ara proves unitàries o canvis en el comportament de l'aplicació.

```
public class Program
{
    public static void Main(string[] args)
    {
        //for production
        Student std1 = new Student(new StudenRepository );

        //for unit test
        Student std2 = new Student(new TestStudentRepository);
    }
}
```



Seguretat Informatica (SI)

Seguretat en les aplicacions

René Serral (forked from: Davide Careglio)

Pratiques

TurnTiquetDispenser

<https://rb.gy/vgyk5>

<https://github.com/emilybache/Racing-Car-Katas/tree/main/CSharp/TurnTicketDispenser>

<https://rb.gy/1m82o>

<https://github.com/emilybache/Racing-Car-Katas/tree/main/Javascript/turn-ticket-dispenser>

Tire-pressure-monitor-system

<https://rb.gy/opqul>

<https://github.com/emilybache/Racing-Car-Katas/tree/main/CSharp/TirePressureMonitoringSystem>

<https://rb.gy/6gxu3>

<https://github.com/emilybache/Racing-Car-Katas/tree/main/Javascript/tire-pressure-monitoring-system>

